

AI & ML for Robotics — AIMLCZG528

Method-by-Method Notes

Every method: what it is · formulas · algorithm · hand-worked example · comparison · where it fits · ML/DNN/RL

Compiled 09 September 2026

Contents

1	How to use these notes	3
2	State Estimation and Motion Models	4
2.1	Discrete Bayes filter (histogram filter over a finite state space)	4
2.2	Velocity motion model	6
2.3	Odometry motion model	7
2.4	Gaussian and sampling-based motion models	9
3	Sensor / Measurement Models	11
3.1	Beam range-finder model (ray-cast, mixture of four noise sources)	11
3.2	Likelihood field (endpoint) model	13
3.3	Correlation-based (map-matching / scan-matching) model	15
3.4	Feature-based measurement model	17
4	Mapping	19
4.1	Occupancy grid mapping (log-odds, inverse sensor model)	19
4.2	Learned inverse sensor model (logistic regression / neural network)	22
4.3	Naive-Bayes multi-sensor fusion and MAP thresholding	23
4.4	MAP occupancy mapping by hill climbing (forward model)	25
5	Localization	27
5.1	Frame transformation (global map frame $\leftrightarrow$ local robot frame)	27
5.2	Grid-based localization (histogram filter)	29
5.3	Monte Carlo Localization (particle filter)	31
5.4	Augmented MCL	32
5.5	Mixture (dual) MCL	34
6	SLAM	36
6.1	EKF-SLAM	36
6.2	EIF-SLAM (Extended Information Form)	39
6.3	SEIF (Sparse Extended Information Form)	40
6.4	GraphSLAM (pose-graph optimisation)	42
6.5	FastSLAM (Rao–Blackwellised particle filter)	44
6.6	Visual SLAM	46
7	Path Planning I — Foundations, Graph Search, Sampling	49
7.1	Costmaps and path evaluation metrics	49
7.2	Dijkstra’s algorithm	51
7.3	A* search	52

7.4	D* (dynamic A*)	53
7.5	RRT and RRT*	55
7.6	PRM and Batch PRM*	58
7.7	Planner selection — the table to memorise	59
8	Path Planning II — Metaheuristic / Evolutionary Planners	60
8.1	Firefly algorithm	61
8.2	Genetic Algorithm for path planning	63
8.3	Ant Colony Optimisation	65
8.4	Particle Swarm Optimisation	67
8.5	Metaheuristic planners — the comparison to have ready	69
9	Learning-Based Path Planning	70
9.1	Reinforcement learning path planner (Q-learning)	70
10	Errata — slide values that do not survive recomputation	74

1 How to use these notes

Every method in the course gets the same seven slots, always in this order:

1. **What it is** — three or four sentences, no padding.
2. **Canonical formulas** — every symbol named.
3. **Algorithm** — five or six steps, ASCII pseudocode.
4. **Worked example** — sized to finish by hand in under 15 minutes.
5. **Compared with** — what it beats, what beats it, and on what axis.
6. **Best suited when** — the conditions that make it the right pick.
7. **ML / DNN / RL** — what learning replaces or augments, with input/output/learned.

Grounding and provenance

Everything here is traced to one of three sources, and nothing else:

- the 169-slide course deck `AIMLRobots Watemarked.pdf`;
- the 17 recorded contact-session transcripts (25 Apr – 29 Aug 2026);
- the EC3 end-semester paper `AIMLEndSemRegular2026.docx` (rescue-robot use case).

Every numerical result in this document was recomputed independently before it was written down. Where the deck's arithmetic does not survive that check, the correct value is printed and the discrepancy is logged in the Errata section (Section 10). Do not carry a slide number into the exam without checking it against that list.

Notation used throughout

Symbol	Meaning
x_t	robot state (pose) at time t ; planar pose is (x, y, θ)
u_t	control / motion command applied between $t - 1$ and t
z_t	measurement at time t ; z_t^k is the k -th beam of that scan
m	the map; m_i the i -th grid cell or landmark
c_t	correspondence (which landmark produced which measurement)
$bel(x_t)$	belief $p(x_t z_{1:t}, u_{1:t})$ — posterior after the measurement update
$\overline{bel}(x_t)$	prediction $p(x_t z_{1:t-1}, u_{1:t})$ — after motion, before measurement
η	normaliser that makes a belief sum (or integrate) to 1
Σ, Ω	covariance matrix and its inverse, the information matrix $\Omega = \Sigma^{-1}$
ξ	information vector, $\xi = \Omega\mu$
K	Kalman gain
H, G	Jacobians of the measurement and motion models
Q, R	process (motion) noise and measurement noise covariances
$l_{t,i}$	log-odds occupancy of cell i at time t
ℓ	path length; η (in RRT*) the steering step size — context separates the two

The one relationship the whole course hangs on

The Bayes filter

$$bel(x_t) = \eta \underbrace{p(z_t | x_t, m)}_{\text{sensor model}} \int \underbrace{p(x_t | u_t, x_{t-1})}_{\text{motion model}} bel(x_{t-1}) dx_{t-1}$$

Everything in Modules 2–5 is one of three things: a way of writing $p(x_t | u_t, x_{t-1})$ (motion models), a way of writing $p(z_t | x_t, m)$ (sensor models), or a way of *representing and propagating* bel that is actually computable (grid filter, Kalman/EKF, particle filter, information filter, pose

graph). Module 6 then asks a different question — not “where am I” but “where do I go” — and Module 7 replaces the hand-designed answer with a learned one.

The EC3 pattern worth internalising

The end-semester paper put *one* use case — a rescue robot in an earthquake-damaged shopping complex — behind all four questions, and every part was scored on whether the answer was tied to that scenario. Three question shapes recurred:

1. **Run one iteration by hand** (FastSLAM, ACO, RRT*) — show every intermediate value.
2. **Pick one of a pair and justify** (velocity vs odometry, beam vs likelihood field, MCL vs augmented MCL, log-odds vs binary grid) — the scenario always contains one planted clause that rules one option out.
3. **Modify the scenario so that X becomes better than Y** — this is question shape 2 run backwards: negate the ruling-out property realistically.

Slot 5 (*Compared with*) and slot 6 (*Best suited when*) of every method below are written specifically to answer shapes 2 and 3.

2 State Estimation and Motion Models

2.1 Discrete Bayes filter (histogram filter over a finite state space)

1. What it is

The exact, non-approximated recursive solution to the Bayes filter when the state space is a finite set. Belief is stored as one probability per state; each cycle multiplies the belief through a transition table (predict) and then through a likelihood table (update), and renormalises. It is the reference implementation every other filter in the course approximates: the grid localizer is this filter with cells as states, MCL is this filter with samples instead of a table, the Kalman filter is this filter with a Gaussian instead of a table.

2. Canonical formulas

$$\overline{bel}(x_t) = \sum_{x_{t-1}} p(x_t | u_t, x_{t-1}) bel(x_{t-1}) \quad bel(x_t) = \eta p(z_t | x_t) \overline{bel}(x_t)$$

$p(x_t | u_t, x_{t-1})$ is the transition (motion) table — read it as “if I was in x_{t-1} and did u_t , how likely am I now in x_t ”. $p(z_t | x_t)$ is the observation table — “if I really am in x_t , how likely was this reading”. $\eta = 1/\sum_x(\cdot)$ is the reciprocal of the sum of the unnormalised numbers, and is the *only* place the evidence probability $p(z_t)$ enters.

3. Algorithm

Algorithm

```

BayesFilter(bel(x_{t-1}), u_t, z_t):
1. for each state x_t:
2.     bel_bar(x_t) = SUM over x_{t-1} of P(x_t | u_t, x_{t-1}) * bel(x_{t-1})
3. for each state x_t:
4.     bel(x_t) = P(z_t | x_t) * bel_bar(x_t)
5. eta = 1 / SUM over x_t of bel(x_t); bel = eta * bel
6. return bel

```

4. Worked example — the door (course canonical)

Exam Tip / Worked Example

State space $\{\text{is_open}, \text{is_closed}\}$, actions $\{\text{push}, \text{do_nothing}\}$. Prior $\text{bel}(x_0) = (0.5, 0.5)$.

Transition table $p(x_t | u_t, x_{t-1})$: push from open $\rightarrow$ open 1.0; push from closed $\rightarrow$ open 0.8, closed 0.2; do_nothing is the identity.

Observation table $p(z_t | x_t)$: $p(\text{sense_open} | \text{open}) = 0.6$, $p(\text{sense_open} | \text{closed}) = 0.2$.

Step 1 — $u_1 = \text{do_nothing}$, $z_1 = \text{sense_open}$.

Predict: do_nothing is the identity, so $\overline{\text{bel}} = (0.5, 0.5)$.

Update: $0.6 \times 0.5 = 0.30$ (open), $0.2 \times 0.5 = 0.10$ (closed).

$\eta = 1/0.40 = 2.5 \Rightarrow \text{bel}(x_1) = (\mathbf{0.75}, 0.25)$.

Step 2 — $u_2 = \text{push}$, $z_2 = \text{sense_open}$.

Predict: open $= 1.0(0.75) + 0.8(0.25) = 0.95$; closed $= 0(0.75) + 0.2(0.25) = 0.05$.

Update: $0.6 \times 0.95 = 0.57$; $0.2 \times 0.05 = 0.01$.

$\eta = 1/0.58 = 1.7241 \Rightarrow \text{bel}(x_2) = (\mathbf{0.9828}, 0.0172)$.

Reading: two consistent sense_open readings plus one push drive belief from 50% to 98.3%. The push contributed more than the sensor did — it moved the *prediction* from 0.75 to 0.95 before any evidence was applied.

5. Compared with

Comparison

Against	It wins because	It loses because
Kalman filter	handles multimodal, non-Gaussian belief; no linearity assumption	cannot represent a continuous state without discretising it
Particle filter (MCL)	exact, deterministic, reproducible — no sampling variance	cost is $O(X)$ per step and $ X $ explodes with dimension
Grid localization	identical algorithm; grid localization <i>is</i> this filter with cells as states	—

6. Best suited when

The state set is genuinely small and discrete (a door, a topological map of rooms, a kill-chain stage in the cyber analogue), belief may be multimodal, and you need an exact answer you can audit by hand. It is also the right pick whenever the exam says “show the belief update step by step” — there is nothing to approximate.

7. ML / DNN / RL

Architecture / Implementation

Traditional: both tables are hand-authored from datasheets and intuition; they go stale the moment the hardware or environment changes.

ML integration: the observation table is the natural learning target. A classifier trained on (state, raw reading) pairs outputs $p(z_t | x_t)$ directly, so a camera frame or a packet capture can serve as z_t without an analytical likelihood.

Input: raw sensor reading + candidate state. **Output:** a likelihood score per state. **Learnt:** the response of this particular sensor in this particular environment.

RL: when the robot may choose u_t , a policy $\pi(u | \text{bel})$ rewarded by reduction in belief entropy performs *active* state estimation — the deck’s “action recommendation” entry point.

2.2 Velocity motion model

1. What it is

The motion model that takes the commanded translational and rotational velocity $u_t = (v, \omega)$ as the input and predicts the pose that results from holding them for Δt . Because v and ω are constant over the interval, the exact trajectory is a circular arc of radius v/ω , not a straight line — which is precisely why a linear Kalman filter cannot represent it and the EKF exists. Noise enters through four hand-tuned parameters $\alpha_1 \dots \alpha_4$ that scale the variance with speed and turn rate.

2. Canonical formulas

$$\begin{pmatrix} x' \\ y' \\ \theta' \end{pmatrix} = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v}{\omega} \sin \theta + \frac{v}{\omega} \sin(\theta + \omega \Delta t) \\ \frac{v}{\omega} \cos \theta - \frac{v}{\omega} \cos(\theta + \omega \Delta t) \\ \omega \Delta t \end{pmatrix}$$

v/ω is the turning radius; as $\omega \rightarrow 0$ this degenerates and you must fall back to the straight-line form $x' = x + v\Delta t \cos \theta$. Noise: $\hat{v} = v + \varepsilon(\alpha_1 v^2 + \alpha_2 \omega^2)$, $\hat{\omega} = \omega + \varepsilon(\alpha_3 v^2 + \alpha_4 \omega^2)$, so error grows with *both* speed and turn rate — fast turns are the worst case.

3. Algorithm

Algorithm

```
VelocityMotionModel(x_{t-1}, u_t = (v, w), dt):
1. sample v_hat = v + noise(alpha1*v^2 + alpha2*w^2)
2. sample w_hat = w + noise(alpha3*v^2 + alpha4*w^2)
3. r = v_hat / w_hat    // turning radius; special-case w_hat near 0
4. x' = x - r*sin(theta) + r*sin(theta + w_hat*dt)
5. y' = y + r*cos(theta) - r*cos(theta + w_hat*dt)
6. theta' = theta + w_hat*dt + final rotation noise; return (x', y', theta')
```

4. Worked example

Exam Tip / Worked Example

Robot at $x_{t-1} = (0, 0, 0)$. Command $v = 1$ m/s, $\omega = 0.5$ rad/s, held for $\Delta t = 2$ s. Noise-free case.

$$r = v/\omega = 1/0.5 = \mathbf{2 \text{ m}}. \quad \omega \Delta t = 1.0 \text{ rad} = 57.30^\circ.$$

$$x' = 0 - 2 \sin 0 + 2 \sin(1.0) = 2(0.84147) = \mathbf{1.6829}$$

$$y' = 0 + 2 \cos 0 - 2 \cos(1.0) = 2 - 2(0.54030) = \mathbf{0.9194}$$

$$\theta' = 0 + 1.0 = \mathbf{1.0 \text{ rad}}$$

The point of the example: a naive straight-line integration gives $x' = v\Delta t = 2.0$, $y' = 0$. The arc model gives (1.683, 0.919) — an error of 0.94 m in y after a single 2-second command. Over a corridor this is the entire difference between arriving and not.

5. Compared with

Comparison

Against	Prefer velocity model when	Prefer the other when
Odometry model	you must predict <i>before</i> moving (collision checking, planning, simulation); or the wheels cannot be trusted to report ground travel (slip, rubble, being carried)	the motion has already happened and encoders are reliable — odometry is strictly more accurate after the fact
Gaussian / sampling model	—	you need a distribution rather than one predicted pose

6. Best suited when

Commands are known before execution and wheel feedback is unreliable or absent: aerial and underwater vehicles with no odometry at all; any wheeled robot on a slip-prone surface; and any use of the model *inside* a planner, where the question is “what would happen if I issued this command” and no encoder reading exists yet.

EC3 Q4(a) — the planted clause

The rescue-robot scenario says “*wheeled locomotion across rubble*”. That single phrase breaks odometry’s founding assumption — that wheel rotation equals ground travel. Wheels spinning on debris report motion that never happened, so encoder integration accumulates error that no amount of noise inflation fixes honestly. The velocity model is commanded in, not measured out, so rubble does not corrupt its input. **Answer: velocity model.**

7. ML / DNN / RL

Architecture / Implementation

Traditional: circular-arc kinematics with hand-tuned $\alpha_1 \dots \alpha_4$, fixed per robot; fails silently after mechanical wear, a payload change, or a terrain shift.

ML integration: a Gaussian Process or MLP learns $p(x_t \mid u_t, x_{t-1})$ directly from (state, control, next-state) triplets recorded during operation — no α tuning, and the model re-adapts from live data.

Input: $(v, \omega) +$ previous pose (x, y, θ) . **Output:** predicted pose distribution — mean and covariance over the next pose. **Learnt:** how speed, rotation, terrain type and mechanical wear scale the error *for this robot*.

Deck example: the ANYmal quadruped — a GP velocity model re-calibrates itself after a 5 kg payload change with no manual re-tuning.

2.3 Odometry motion model

1. What it is

The motion model that reads the pose change already reported by the wheel encoders and decomposes it into three elementary movements — rotate, translate, rotate — then applies those to the true pose with noise on each. It is a *measurement* of motion, not a command, so it can only be used after the robot has moved. In practice it beats the velocity model on accuracy whenever the wheels are honest, which is why the textbook treats odometry as control data even though the encoder is physically a sensor.

2. Canonical formulas

Given the odometry pair $\bar{x}_{t-1} = (\bar{x}, \bar{y}, \bar{\theta})$ and $\bar{x}_t = (\bar{x}', \bar{y}', \bar{\theta}')$:

$$\delta_{rot1} = \text{atan2}(\bar{y}' - \bar{y}, \bar{x}' - \bar{x}) - \bar{\theta}, \quad \delta_{trans} = \sqrt{(\bar{x}' - \bar{x})^2 + (\bar{y}' - \bar{y})^2},$$

$$\delta_{rot2} = \bar{\theta}' - \bar{\theta} - \delta_{rot1}$$

and the true pose advances as

$$x' = x + \delta_{trans} \cos(\theta + \delta_{rot1}), \quad y' = y + \delta_{trans} \sin(\theta + \delta_{rot1}), \quad \theta' = \theta + \delta_{rot1} + \delta_{rot2}.$$

The three δ terms carry independent noise $\hat{\delta} = \delta - \varepsilon(\alpha\delta^2 + \alpha'\delta'^2)$: turn error scales with turning, translation error with distance.

3. Algorithm

Algorithm

```
OdometryMotionModel(x_{t-1}, u_t = (xbar_{t-1}, xbar_t)):
1. d_rot1 = atan2(ybar' - ybar, xbar' - xbar) - thetabar
2. d_trans = sqrt((xbar' - xbar)^2 + (ybar' - ybar)^2)
3. d_rot2 = thetabar' - thetabar - d_rot1
4. subtract sampled noise from each of d_rot1, d_trans, d_rot2
5. x' = x + d_trans*cos(theta + d_rot1); y' = y + d_trans*sin(theta + d_rot1)
6. theta' = theta + d_rot1 + d_rot2; return (x', y', theta')
```

4. Worked example

Exam Tip / Worked Example

Odometry reports the robot moved from $(2, 3, 0^\circ)$ to $(4, 6, 30^\circ)$ in its own internal frame. The robot's true pose (from the filter) is $(0, 0, 0^\circ)$.

Decompose:

$$\delta_{trans} = \sqrt{(4-2)^2 + (6-3)^2} = \sqrt{4+9} = \sqrt{13} = \mathbf{3.6056}$$

$$\delta_{rot1} = \text{atan2}(3, 2) - 0 = \mathbf{56.31^\circ}$$

$$\delta_{rot2} = 30^\circ - 0^\circ - 56.31^\circ = \mathbf{-26.31^\circ}$$

Apply to the true pose:

$$x' = 0 + 3.6056 \cos(56.31^\circ) = 3.6056(0.5547) = \mathbf{2.0}$$

$$y' = 0 + 3.6056 \sin(56.31^\circ) = 3.6056(0.8321) = \mathbf{3.0}$$

$$\theta' = 0 + 56.31^\circ - 26.31^\circ = \mathbf{30^\circ}$$

Sanity check that is worth doing in the exam: the odometry frame and the true frame started aligned here, so the answer must equal the raw odometry *delta* $(+2, +3, +30^\circ)$, and it does. If the true θ had been, say, 90° , the same three δ values would rotate the displacement into a completely different place — that is the whole reason for the rotate–translate–rotate decomposition rather than just adding $(\Delta x, \Delta y)$.

5. Compared with

Comparison

More accurate than the velocity model *after the fact*, because it uses what the wheels actually did rather than what was asked of them. Strictly worse than the velocity model whenever the wheel–ground contract is broken (slip, rubble, carpet-to-tile transitions, payload shifts, being lifted). Cannot be used at all inside a planner, since it needs a completed motion as input.

6. Best suited when

Flat, high-friction, indoor floors; a differential-drive base with good encoders; and a filter that is correcting the estimate after each move rather than predicting ahead of it. Warehouse AMRs on sealed concrete are the canonical fit.

7. ML / DNN / RL

Architecture / Implementation

Traditional: dead-reckoning by integrating encoder readings with a fixed Gaussian noise model — catastrophically wrong on slip-prone or irregular terrain.

ML integration: a CNN slip predictor ingests a downward terrain image plus an IMU reading before each step and outputs a slip-ratio correction applied to the encoder deltas *before* pose integration.

Input: encoder deltas (Δ_L, Δ_R) + IMU acceleration + terrain image. **Output:** slip-corrected pose update $(\Delta x, \Delta y, \Delta \theta)$ plus a per-step confidence. **Learnt:** surface-specific slip ratios, and the correlation between IMU spike patterns and encoder error.

Deck example: NASA JPL terrain-adaptive odometry for the Perseverance rover — trained on Earth, deployed on Mars.

2.4 Gaussian and sampling-based motion models

1. What it is

Two ways of turning a point prediction into a distribution. The Gaussian model assumes the displacement is unimodal and adds $\mathcal{N}(\mu, \sigma^2)$ noise, so a single sample — the mean — carries the answer and a covariance carries the spread; this is what a Kalman filter consumes. The sampling-based model drops the unimodality assumption entirely, drawing n weighted hypotheses from the noise distribution; this is what a particle filter consumes. The deck also lists vision- and language-derived motion models, where u_t is inferred from images or from a natural-language instruction rather than read off a controller.

2. Canonical formulas

Gaussian form: $p(x_t \mid u_t, x_{t-1}) = \mathcal{N}(x_t; g(u_t, x_{t-1}), R_t)$ where g is the deterministic velocity or odometry map above and R_t its noise covariance. Sampling form: draw $x_t^{[i]} \sim p(x_t \mid u_t, x_{t-1})$ for $i = 1 \dots M$, giving an unweighted particle set that approximates the same density. For a scalar, $\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$.

3. Algorithm

Algorithm

```
SampleMotionModel(x_{t-1}, u_t, M):
1. compute the deterministic prediction g(u_t, x_{t-1})
2. for i = 1..M:
3.     draw noise e_i from N(0, R_t) // or from the alpha-scaled per-component noise
4.     x_t^{[i]} = g(u_t, x_{t-1}) + e_i
5. return the particle set {x_t^{[1]}, ..., x_t^{[M]}}
// Gaussian variant: return (g(u_t, x_{t-1}), R_t) instead of samples
```

4. Worked example

Exam Tip / Worked Example

Commanded straight-line motion of 1.0 m along $\theta = 0$, with translational noise $\sigma = 0.1$ m. Robot at $(0, 0, 0)$.

Gaussian model: predicted pose $\mathcal{N}((1.0, 0, 0), \text{diag}(0.01, \dots))$. A single number plus a variance; one hypothesis only.

Sampling model, $M = 4$, with noise draws $(+0.08, -0.12, +0.03, -0.05)$: particles at $x = 1.08, 0.88, 1.03, 0.95$. Sample mean $= (1.08 + 0.88 + 1.03 + 0.95)/4 = \mathbf{0.985}$, sample variance $= \frac{1}{3} \sum (x_i - 0.985)^2 = \frac{1}{3}(0.009025 + 0.011025 + 0.002025 + 0.001225) = \mathbf{0.0078}$, i.e. $\hat{\sigma} = 0.088$ against the true 0.10.

Reading: with only 4 particles the estimated spread is already within 12% of the truth, but the mean is off by 1.5 cm — sampling error. That trade (unbiased shape, noisy estimate) is exactly what you buy when you move from Kalman to MCL.

5. Compared with**Comparison**

Gaussian model: cheap, one hypothesis, feeds KF/EKF, collapses catastrophically if the true posterior is bimodal. Sampling model: represents any shape including multimodal, feeds MCL and FastSLAM, costs M evaluations per step and introduces sampling variance. The deck's rule: Gaussian when uncertainty is limited and real-time tracking is not needed; sampling when multiple hypotheses genuinely exist and compute is available.

6. Best suited when

Gaussian: pose tracking from a known start, tight uncertainty, limited compute. Sampling: global localization, kidnapped-robot recovery, or any time the answer “I am 70% here and 25% over there” has to survive the next step instead of being averaged away.

7. ML / DNN / RL**Architecture / Implementation**

Traditional: Monte Carlo sampling using the analytic velocity model as the proposal — needs 10,000+ particles in complex environments, prohibitively slow in real time.

ML integration: a neural proposal $q(x_t | x_{t-1}, z_t, u_t)$ places particles exactly where the posterior has mass; a *differentiable* particle filter trains the proposal and the sensor model jointly end-to-end.

Input: previous particle set + u_t + observation z_t (image or LiDAR scan) + map. **Output:** weighted particle set — the full non-Gaussian posterior. **Learnt:** a smart proposal distribution and a discriminative sensor likelihood.

Deck example: NVIDIA's differentiable particle filter — 100× fewer particles at equal localization accuracy for autonomous vehicles.

Vision / language variants: CNNs (RAFT, FlowNet) and transformers (SuperGlue, LoFTR) replace optical flow and feature matching, so u_t comes from consecutive RGB frames (Tesla Autopilot, DeepVO); an LLM + skill library + diffusion policy turns “go to the kitchen counter” into u_t directly (Google RT-2, SayCan).

3 Sensor / Measurement Models

The one distinction to get right first

$p(z | x)$ is the **measurement (forward) model** — “given that I am here, how likely was this reading?” It is generative, it is what the physics gives you, and it is what all four models below compute. $p(x | z)$ is the **posterior** — “given this reading, where am I?” It is discriminative and you only get it through Bayes’ rule, using $p(z | x)$ as the input. The deck’s spam analogy: $p(z | x)$ is “how likely does a spam email contain the word lottery”; $p(x | z)$ is “how likely is this email spam given that it says lottery”.

3.1 Beam range-finder model (ray-cast, mixture of four noise sources)

1. What it is

A physics-first model. For each beam it ray-casts through the map from the hypothesised pose to find the expected range z^{k*} , compares it to the measured range z^k , and scores the difference under a four-component mixture that accounts for normal noise, unexpected obstacles, detection failure and random junk. Beams are treated as conditionally independent, so the scan likelihood is a product over beams.

2. Canonical formulas

$$p(z_t | x_t, m) = \prod_{k=1}^K p(z_t^k | x_t, m)$$

$$p(z_t^k | x_t, m) = w_{hit} p_{hit} + w_{short} p_{short} + w_{max} p_{max} + w_{rand} p_{rand}, \quad \sum_i w_i = 1$$

with $p_{hit} = \mathcal{N}(z^k; z^{k*}, \sigma_{hit}^2)$ truncated to $[0, z_{max}]$; $p_{short} = \lambda_{short} e^{-\lambda_{short} z^k}$ for $z^k < z^{k*}$ (someone stepped in front of the beam); p_{max} a point mass at z_{max} (nothing came back); $p_{rand} = 1/z_{max}$ uniform (unexplainable). The intrinsic parameters $\sigma_{hit}, \lambda_{short}, w_i$ are fitted from logged data by MLE / EM.

3. Algorithm

Algorithm

```
BeamRangeFinderModel(z_t, x_t, m):
1.  q = 1
2.  for k = 1..K:
3.      z_star = RayCast(x_t, beam angle k, m)  // expected range from the map
4.      p = w_hit * P_hit(z^k | z_star) + w_short * P_short + w_max * P_max +
        w_rand * P_rand
5.      q = q * p  // conditional independence across beams
6.  return q
```

4. Worked example

Exam Tip / Worked Example

(a) **Single-beam Gaussian hit.** Robot hypothesised at $x = 6$ m, map says the wall is at

10 m, so ray-casting gives $z^* = 10 - 6 = 4$ m. The laser returns $z = 3.9$ m.

$$e = 3.9 - 4.0 = -0.1, \quad \exp\left(-\frac{e^2}{2\sigma^2}\right) = \exp\left(-\frac{0.01}{2(0.04)}\right) = \exp(-0.125) = \mathbf{0.8825}$$

(with $\sigma = 0.2$). A high score — the reading is consistent with this pose.

(b) Full mixture. $z^* = 5$, $z = 4.7$, weights $w_{hit} = 0.7$, $w_{short} = 0.1$, $w_{max} = 0.1$, $w_{rand} = 0.1$ and component values $p_{hit} = 0.85$, $p_{short} = 0.3$, $p_{max} = 0$, $p_{rand} = 0.05$:

Component	w_i	p_i	$w_i p_i$
hit	0.7	0.85	0.595
short	0.1	0.30	0.030
max	0.1	0.00	0.000
rand	0.1	0.05	0.005
$p(z \mid x, m)$			0.630

The hit component supplies 94% of the mass, so this scan strongly supports the pose — but note that the 0.005 floor from p_{rand} is what stops a single freak reading from driving the whole product to zero.

5. Compared with

Comparison

Against	Beam model wins because	Beam model loses because
Likelihood field	it validates the <i>whole ray</i> , so it can tell “wall at 4 m” from “wall at 4 m with a hole in front” and is physically interpretable with no training	ray-casting is $O(\text{range})$ per beam and the likelihood is highly non-smooth in pose — tiny pose changes flip which cell the ray hits
Correlation model	it is per-beam, so a single bad beam costs only its own factor	correlation needs no ray-cast at all
Feature-based	it needs no landmarks, so it works in featureless corridors	it throws away semantics; identical corridors stay ambiguous

Limitation / Warning

Three named failure modes: **non-smoothness** (the likelihood is jagged in pose, so gradient-based pose search gets stuck); **dynamic sensitivity** (a person standing in the beam produces a short reading that the model can only absorb through p_{short}); **computation** (ray-casting every beam for every particle is the dominant cost in MCL). And the product over K beams means a handful of wrong z^* values can veto an otherwise correct pose.

6. Best suited when

The map geometry is fixed and trustworthy, the environment is static, causality and explainability are required, and there is no training data. The deck’s example: precise localization in a static warehouse with known geometry but no visual landmarks (reflective metal racks), where feature models fail on perceptual aliasing in repetitive aisles and DNNs would need re-labelling for every layout change.

EC3 Q4(b) — why the beam model was the wrong answer there

The rescue scenario has a *pre-disaster* map (so z^* from ray-casting is systematically wrong wherever a wall has collapsed) and *intermittent smoke* (so beams are blocked at random). Both attack the beam model at its weakest point: the product over beams lets a few wrong z^* values veto the correct pose. The likelihood field scores only where the endpoint lands and degrades gracefully instead. **Answer: likelihood field.**

7. ML / DNN / RL**Architecture / Implementation**

Noise parameters (σ, λ, w_i) — **classical ML (EM)**: iteratively fits the Gaussian and exponential parameters to historical sensor logs. Deck example: indoor office navigation, learning the LiDAR noise profile against glass walls versus drywall and refining the random-hit weight to suppress phantom obstacles.

Beam likelihood — **DNN**: learns non-linear likelihoods straight from raw point-cloud or pixel data. Deck example: crowded malls, where a CNN predicts which beams belong to moving humans so the bot scores only the static pillars.

Subsampling rate — **RL**: a policy chooses *which* beams to process to maximise information gain per CPU cycle. Deck example: UAV high-speed mapping — thin the beams in uniform corridors, thicken them at intersections.

3.2 Likelihood field (endpoint) model**1. What it is**

An outcome-only model. It ignores the path the beam travelled and asks a single question of each measurement: after projecting the reading into the map frame, how far is the endpoint from the nearest obstacle? That distance is looked up in a pre-computed distance field, so the cost per beam is a constant-time array read instead of a ray-cast. The resulting likelihood is smooth in pose, which is exactly what the beam model is not.

2. Canonical formulas

Project the endpoint into the map frame:

$$x_{end} = x + z^k \cos(\psi + \theta_k), \quad y_{end} = y + z^k \sin(\psi + \theta_k)$$

where (x, y, ψ) is the robot pose and θ_k the beam's bearing in the robot frame. Then, with d the distance-field value at that endpoint cell,

$$p(z^k \mid x, m) = \exp\left(-\frac{d^2}{2\sigma^2}\right)$$

and the scan likelihood is again the product over beams. In practice the same $w_{hit}/w_{max}/w_{rand}$ mixture is layered on top; p_{short} has no meaning here because there is no ray to be cut short.

3. Algorithm**Algorithm**

```
LikelihoodFieldRangeFinder(z_t, x_t, m):
0. (offline, once) build distance field D: D[i][j] = distance from cell to
   nearest obstacle
```

```

1. q = 1
2. for k = 1..K with z^k != z_max:
3.     x_end = x + z^k*cos(psi + theta_k); y_end = y + z^k*sin(psi + theta_k)
4.     d = D[cell(x_end, y_end)]    // constant-time lookup, no ray casting
5.     q = q * ( w_hit*exp(-d^2 / (2*sigma^2)) + w_rand/z_max )
6. return q

```

4. Worked example

Exam Tip / Worked Example

Robot pose $(x, y, \psi) = (10.0, 5.0, 0)$. One beam: $z^k = 3.92$ m at $\theta_k = 0.523$ rad (30°). Distance-field noise $\sigma = 0.25$ m.

Project: $x_{end} = 10 + 3.92 \cos(0.523) = 10 + 3.92(0.8664) = \mathbf{13.40}$, $y_{end} = 5 + 3.92 \sin(0.523) = 5 + 3.92(0.4993) = \mathbf{6.96}$.

Look up and score a five-point cloud (each row: read d from the field, then $\exp(-d^2/2\sigma^2)$ with $\sigma = 0.25$, i.e. $2\sigma^2 = 0.125$):

#	z^k	d (m)	$p = \exp(-d^2/0.125)$
1	3.92	0.08	0.9501
2	4.05	0.12	0.8912
3	3.88	0.05	0.9802
4	2.10	2.40	≈ 0
5	4.01	0.10	0.9231

Four endpoints snap to within 12 cm of a mapped obstacle and score high; point 4 lands 2.4 m from anything and is effectively rejected.

Errata warning: the deck prints these scores against “ $\sigma = 0.1$ ”. With $\sigma = 0.1$ the first row evaluates to $\exp(-0.0064/0.02) = 0.726$, not 0.96. The printed score column is self-consistent only for $\sigma \approx 0.25$, which is the value used above. See Section 10. In an exam, **state the σ you are using and show the exponent** — that protects the marks regardless.

5. Compared with

Comparison

Axis	Beam (ray-cast)	Likelihood field (endpoint)
Question asked	“was the journey clear?”	“how close is the result to a wall?”
Cost per beam	$O(N)$, N = range in cells	$O(1)$ lookup — the deck quotes 500× faster, 100k+ points/sec
Smoothness in pose	jagged; hostile to gradient search	smooth; friendly to scan matching
Robustness	fragile to map pixel errors	snaps the cloud to the map, tolerates noise
Physical fidelity	correct — knows about occlusion	cannot see through walls: an endpoint behind a wall scores as well as one in front

Limitation / Warning

Three limitations: **dynamic obstacles** (endpoints land on people not in the static map, dragging the pose estimate); **outlier sensitivity** (one endpoint near a spurious mapped obstacle

inflates the score); **discretisation error** (the field is only as good as the cell size). And the structural one: because occlusion is ignored, a pose that is physically impossible can still score well.

6. Best suited when

High-entropy, unstructured environments with no reliable discrete landmarks — dense foliage, shifting debris — and any situation where the map is stale, partly wrong, or partly occluded, because scoring only the endpoint degrades gracefully where ray-casting fails outright. Also whenever the pose search is an optimisation (scan matching), since it needs a smooth objective.

7. ML / DNN / RL

Architecture / Implementation

Noise parameter σ — classical ML (EM): fits the endpoint noise spread to historical logs. Deck example: ultrasound SLAM in medical imaging, learning tissue-specific speckle noise so the likelihood peaks sharpen at anatomical boundaries.

Measurement decoding — DNN (PointNet++): classifies each endpoint semantically before it is scored. Deck example: forestry drones — classify endpoints as “foliage” (soft noise) or “trunk” (hard landmark) so the likelihood only trusts trunks.

Scan-matching policy — RL: scales the search resolution with pose uncertainty. Deck example: AUVs — subsample endpoints over flat seabed to save power, intensify over coral reefs to prevent drift.

Hybrid fix for the dynamic-obstacle limitation: a DNN labels endpoints static vs. dynamic and drops the dynamic ones before the likelihood field ever sees them.

3.3 Correlation-based (map-matching / scan-matching) model

1. What it is

Treats the local sensor scan as a small binary occupancy image and scores how well it overlays the stored map at the hypothesised pose. There is no per-beam physics and no landmark extraction — just cell-by-cell agreement, normalised into a likelihood. It is the model behind classical scan matching, where a hill-climbing search over pose maximises this overlap.

2. Canonical formulas

Overlap form:

$$p(z \mid x, m) = \eta \sum_i (m_i \cap z_i)$$

counting cells where map and scan are both occupied, with η a normaliser. Product-of-agreements form (the one that is actually computed):

$$p(Z \mid X, m) = \prod_i p(z_i \mid x, m_i), \quad p(z_i) = \begin{cases} p_{agree} & \text{cell matches} \\ 1 - p_{agree} & \text{cell mismatches} \end{cases}$$

3. Algorithm

Algorithm

```
CorrelationModel(z_t, x_t, m):
1. transform the local scan into the map frame at pose x_t
```

```

2. overlay the scan window on the stored map, cell by cell
3. for each cell i: score_i = p_agree if scan(i) == map(i) else (1 -
p_agree)
4. q = PRODUCT of score_i over the window // or eta * SUM of (m_i AND z_i)
5. (scan matching) hill-climb over candidate poses x_t, keep the argmax of q
6. return q

```

4. Worked example

Exam Tip / Worked Example

A 3×3 local window, 9 cells. Agreement probability $p_{agree} = 0.9$, so a mismatch scores 0.1.
Scan A — perfect overlay, 9 matches:

$$p(Z | X, m) = 0.9^9 = \mathbf{0.3874}$$

Scan B — 8 matches, 1 mismatch (the sensor sees an obstacle where the map says free):

$$p(Z | X, m) = 0.9^8 \times 0.1 = 0.4305 \times 0.1 = \mathbf{0.04305}$$

Likelihood ratio = $0.3874/0.04305 = \mathbf{9.0}$ — exactly $p_{agree}/(1 - p_{agree})$, which is worth remembering: *each mismatching cell costs a factor of 9.*

The lesson the deck draws: this is graceful degradation. Had the model used a hard 0/1 agreement instead of 0.9/0.1, one mismatch would send the whole product to zero — the “likelihood collapse” failure. A single person standing in a hallway would make the robot’s true pose impossible.

5. Compared with

Comparison

Against the beam model: no ray-casting, so much cheaper per pose, but it throws away occlusion reasoning and needs a dense local map. Against the likelihood field: comparable cost but works on raw occupancy rather than a distance transform, and is more sensitive to discretisation. Against feature-based: works where there are no features at all, but cannot disambiguate two geometrically identical places.

Limitation / Warning

Named failure modes: **local minima** (hill-climbing over pose converges to the wrong alignment); **symmetry ambiguity** (“infinite hallways” — every pose along the corridor overlays equally well); **computational cost** (searching a dense pose grid); and **likelihood collapse** if hard agreement is used instead of soft.

6. Best suited when

Featureless but geometrically rich environments — long corridors with side structure, warehouse aisles — where beams are cheap to gather but no landmark survives, and where you are refining an already-close pose rather than searching globally.

7. ML / DNN / RL

Architecture / Implementation

Normaliser η — classical ML (regression): learns the environment-specific scaling that prevents likelihood collapse. Deck example: warehouse AMRs — analyse historical occupancy drift so a minor shelving shift does not invalidate the whole match score.

Dynamic segmentation — DNN (CNN / U-Net): denoises the scan before matching. Deck example: hospital service robots — a U-Net segments humans and gurneys out of the 2-D LiDAR slice so only stable hallway geometry is matched.

Search step — RL: an adaptive-resolution policy driven by environmental entropy. Deck example: last-mile delivery — coarse search on featureless sidewalks to save power, fine search at complex intersections.

End-to-end: neural likelihood fields predict the correlation likelihood directly from raw scans, skipping the overlay entirely.

3.4 Feature-based measurement model

1. What it is

Abstracts the raw scan into a handful of geometric primitives — corners, lines, doors, tags — matches each observed feature to a mapped landmark, and scores the match by how far the observation is from the prediction *in units of its own uncertainty*. That last part is the Mahalanobis distance, and the same quantity does double duty as a rejection gate. This is the measurement model that EKF-SLAM assumes.

2. Canonical formulas

Squared Mahalanobis distance between an observed feature f and a mapped landmark m :

$$d^2 = (f - m)^T \Sigma^{-1} (f - m)$$

Σ is the innovation covariance, so d^2 is a χ^2 variable: distance scaled by how uncertain you were. Likelihood and gate:

$$p(f | x, m) \propto \exp\left(-\frac{1}{2}d^2\right), \quad \text{accept the correspondence iff } d^2 < \chi_{\text{gate}}^2$$

Scan likelihood is the product over accepted correspondences, $p(z | x, m) = \prod_i p(f_i | x, m)$. Standard 95% gates: $\chi^2 = 3.84$ for 1 d.o.f., 5.99 for 2 d.o.f.

3. Algorithm

Algorithm

```
FeatureBasedModel(z_t, x_t, m):
1.  extract features f_1..f_n from the raw scan (corners, lines, tags)
2.  for each f_i: predict where each mapped landmark should appear from pose x_t
3.      compute d^2 = (f_i - m_j)^T * Sigma^-1 * (f_i - m_j) for each candidate j
4.      pick j* = argmin d^2; reject the feature entirely if d^2 > chi2 gate
5.      p_i = exp(-0.5 * d^2)
6.  return PRODUCT of p_i over accepted correspondences
```

4. Worked example

Exam Tip / Worked Example

Two observed 2-D features, identity covariance $\Sigma = I$ (so the Mahalanobis distance is just the Euclidean distance squared), gate $\chi^2 = 5.99$.

Feature 1: observed $f_1 = [2, 1]$, mapped landmark $m_1 = [2, 1]$.

$$d_1^2 = (0)^2 + (0)^2 = 0 \Rightarrow p_1 = e^{-0} = \mathbf{1.00}$$

Feature 2: observed $f_2 = [5, 5]$, mapped landmark $m_2 = [6, 5]$.

$$d_2^2 = (5 - 6)^2 + (5 - 5)^2 = 1 \Rightarrow p_2 = e^{-0.5} = \mathbf{0.6065}$$

Both pass the gate ($0 < 5.99$, $1 < 5.99$), so

$$p(z \mid x, m) = 1.00 \times 0.6065 = \mathbf{0.6065}$$

Now change one thing. Suppose feature 2's sensor is far less certain in x : $\Sigma = \text{diag}(4, 1)$. Then $d_2^2 = 1/4 + 0 = 0.25$ and $p_2 = e^{-0.125} = 0.8825$. *Identical geometry, better score* — because a 1 m error is unremarkable when your σ_x is 2 m. That is the whole reason to use Mahalanobis rather than Euclidean distance, and it is a one-line answer worth having ready.

5. Compared with**Comparison**

Far more compact than beam/likelihood-field/correlation — the state carries a few landmarks instead of a dense grid, which is what makes EKF-SLAM tractable at all. Robust to global scale shifts and to irrelevant structural change, because it ignores empty space. But it requires that stable, extractable features actually exist, and it inherits the **data association** problem in full: the wrong correspondence corrupts the map permanently.

Limitation / Warning

Perceptual aliasing is the defining limitation: in a symmetric building, observation $\neq$ unique landmark. Four standard association strategies, weakest first — nearest neighbour (fast, naive); Mahalanobis gating (the EKF standard); joint compatibility (checks all observations for mutual consistency, most robust); visual descriptors (ORB / SIFT / BRIEF, used in visual SLAM).

6. Best suited when

Sparse, semantically distinct landmarks exist and long-range navigation matters: outdoor landmark navigation, AprilTag-instrumented warehouses, an intact information kiosk in an otherwise wrecked building. It is also the only one of the four models that scales to large maps without the state exploding.

7. ML / DNN / RL**Architecture / Implementation**

Gate threshold — classical ML (MLE): learns the χ^2 gate from sensor-noise logs rather than assuming it. Deck example: smart-office navigation — adapt the gate to the sensor variance seen in narrow corridors so similar doorway corners are not confused.

Semantic extraction — DNN (YOLO / SSD): identifies and classifies landmark types robustly under changing light. Deck example: retail inventory robots — YOLO reads aisle

markers and shelf headers as named landmarks, giving a semantic map more robust than raw geometry.

Feature-selection policy — RL: learns which landmarks give the most information gain per compute cycle. Deck example: delivery drones — ignore transient foliage, track stable rooflines and street signs.

Sensor-model chooser — one line each

Beam: static, trusted map, need physics and explainability, can afford ray-casts.

Likelihood field: stale or cluttered map, dynamic scene, need speed and a smooth objective, willing to give up occlusion reasoning.

Correlation: dense geometry, no landmarks, refining an already-close pose.

Feature-based: sparse distinctive landmarks, large map, need a compact state — and you have a plan for data association.

4 Mapping

Inverse vs forward model — the axis this whole module turns on

Inverse model $p(m \mid z, x)$: “the sensor read z ; what is the probability there is a wall at D ?” One cell at a time, discriminative, the analogue of a classifier. This is what occupancy grid mapping uses.

Forward model $p(z \mid m, x)$: “given this hypothetical map, how likely am I to see these measurements?” Whole map at once, generative. This is what MAP / hill-climbing mapping uses.

The deck’s phrasing: inverse asks “what does this single pixel look like given this ray”; forward asks “what is the most likely configuration of the *entire world* that explains all my sensor data together”.

4.1 Occupancy grid mapping (log-odds, inverse sensor model)

1. What it is

Assumes known poses, discretises the world into cells, and treats each cell as an independent binary random variable. Because the cells are independent, the recursive Bayes update for each cell reduces to *addition* in log-odds space — which is why it runs on embedded hardware. Each scan adds a positive increment to cells at the hit and a negative increment to cells the beam passed through, and a sigmoid converts back to probability whenever the map is queried.

2. Canonical formulas

Log-odds and its inverse:

$$l(p) = \log \frac{p}{1-p}, \quad p = 1 - \frac{1}{1+e^l} = \frac{1}{1+e^{-l}}$$

Recursive update for cell i :

$$l_{t,i} = l_{t-1,i} + \underbrace{\log \frac{p(m_i \mid z_t, x_t)}{1 - p(m_i \mid z_t, x_t)}}_{\text{inverse sensor model}} - \underbrace{l_{0,i}}_{\text{prior, subtracted so it is not double-counted}}$$

With an uninformative prior $p = 0.5$, $l_0 = 0$ and the term drops out — which is why the deck's grids all start at 0. Summing log-odds is exactly equivalent to multiplying probabilities by Bayes' rule, but with no division, no normalisation and no underflow.

3. Algorithm

Algorithm

```
OccupancyGridMapping({l_{t-1,i}}, x_t, z_t):
1. for all cells m_i:
2.     if m_i is inside the sensor cone of z_t:
3.         l_{t,i} = l_{t-1,i} + inverse_sensor_model(m_i, x_t, z_t) - l_0
4.     else:
5.         l_{t,i} = l_{t-1,i}    // untouched cells keep their value
6. return {l_{t,i}}    // convert with p = 1/(1+exp(-l)) only when the map is
                        queried
```

4. Worked example

Exam Tip / Worked Example

Setup (the course's 2×2 grid). Prior $p(\text{occupied}) = 0.5$ for every cell, so $l_0 = 0$. Sensor: $p(\text{Detect} \mid \text{Occupied}) = 0.8$, $p(\text{Detect} \mid \text{Free}) = 0.2$.

Inverse-model increments (compute these once, reuse forever):

$$l_{occ} = \log \frac{0.8}{0.2} = \log 4 = +1.3863, \quad l_{free} = \log \frac{0.2}{0.8} = -1.3863$$

Scan 1 — robot detects an obstacle in C1, passes through C2.

C1: $l = 0 + 1.3863 - 0 = 1.3863 \Rightarrow p = 1/(1 + e^{-1.3863}) = 0.80$

C2: $l = 0 - 1.3863 - 0 = -1.3863 \Rightarrow p = 0.20$

C3, C4: untouched, $l = 0$, $p = 0.50$

Scan 2 — C1 detected again.

$l = 1.3863 + 1.3863 = 2.7726 \Rightarrow p = 1/(1 + e^{-2.7726}) = 0.9412$

Scan 3 — C1 now reads free (a person walked through, or noise).

$l = 2.7726 - 1.3863 = 1.3863 \Rightarrow p = 0.80$

The property to name in an exam: belief moved $0.5 \rightarrow 0.80 \rightarrow 0.94 \rightarrow 0.80$. One contradicting reading *dents* the estimate but does not erase two scans of evidence. A deterministic binary grid would have flipped the cell to free and thrown all history away.

5. Compared with

Comparison

Against	Log-odds grid wins because	It loses because
Binary (deterministic) grid	keeps a confidence, so transient obstacles decay instead of becoming permanent; evidence accumulates	needs a float per cell and a sigmoid on query
Feature-based map	needs no landmarks; represents free space explicitly, which is what a planner actually wants	memory is $O(\text{area}/\text{resolution}^2)$; no semantics
SLAM	far cheaper — no joint pose-map estimation	requires known poses ; drift in x_t smears the map irreparably

Limitation / Warning

The independence assumption is false and the deck says so. Treating each cell separately maps two overlapping beams into conflicting probabilities in the overlap region. A sensor reading only requires an obstacle *somewhere* in the measurement cone, not everywhere in it — there exist maps that explain the scan with no conflict at all, and the factorised update cannot find them. That is exactly the gap the forward-model / hill-climbing method below fills. Two further named challenges: **data association** (pose drift makes scans land on the wrong historical feature — “edge smearing”), and **residual uncertainty** (unmeasured drift from slippage or payload shifts, mitigated by covariance inflation, loop closure and temporal smoothing).

6. Best suited when

Poses are already known or reliably estimated, the planner needs explicit free space, and the environment is dynamic enough that you need confidence rather than a hard 0/1 label.

EC3 Q4(d) and Q1(b)(ii) — both answered from this method

Q4(d), log-odds vs binary grid. The rescue scenario has *intermittent smoke*. A binary grid writes the smoke in as a permanent wall the first time a beam is blocked, and the robot then plans around a corridor that is actually clear. Log-odds lets the smoke cell decay back toward 0.5 as later clear scans arrive. **Answer: Bayesian log-odds.**

Q1(b)(ii), when occupancy grid mapping beats SLAM. SLAM exists because the pose is unknown. Modify the scenario so it is known — rescue workers have laid retro-reflective survey markers on a surveyed grid, or the robot is tethered on a measured cable, or it works a single room already registered to the pre-disaster map. Localization is then solved externally, and all that remains is “what has changed in this room”, which is precisely $p(m \mid z, x)$ with x given. Mapping alone suffices, at a fraction of the compute.

7. ML / DNN / RL

Architecture / Implementation

The inverse sensor model itself is the learning target — see the next two methods. Deck use cases where a hand-written inverse model fails and a learned one is required: **glass doors** (LiDAR passes through; a 1-D CNN on RGB-D recognises reflection patterns and outputs $p = 0.95$ occupied); **steam and dust** (a deep net on LiDAR intensity marks low-intensity scatter as

$p = 0.05$, free); **cyber port mapping** (an MLP on packet timing scores vulnerability); **FinOps** (an RNN on time-series usage separates genuine waste from backup routines).

4.2 Learned inverse sensor model (logistic regression / neural network)

1. What it is

Replaces the hand-written rule “before the hit is free, at the hit is occupied” with a classifier trained on real sensor data against a ground-truth map. Features are extracted from the raw reading — normalised range, return intensity — and a logistic (or deeper) model outputs $p(m_i | z, x)$ directly. Its output then feeds the *same* log-odds recursion as before, so the mapping machinery is unchanged; only the increment is learned.

2. Canonical formulas

Forward pass:

$$a = b + w_1 x_1 + w_2 x_2, \quad p(\text{occ}) = \sigma(a) = \frac{1}{1 + e^{-a}}$$

Binary cross-entropy loss and its gradient:

$$L = -[m \log \hat{p} + (1 - m) \log(1 - \hat{p})], \quad \frac{\partial L}{\partial w_j} = (\hat{p} - m) x_j$$

Weight update: $w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial L}{\partial w}$. Then the map recursion, unchanged: $L_t = L_{t-1} + \text{logit}(p_t) - L_0$.

Study Note

Note the elegant identity the deck relies on: $\text{logit}(\sigma(a)) = a$. So the network’s pre-activation a is the log-odds increment — you never have to run the sigmoid and then take the logit back. In the deployment trace below, the a values are simply summed.

3. Algorithm

Algorithm

```

LearnedInverseModel:
1. (lab) drive the robot in a room with a ground-truth map from
   high-precision LiDAR
2. (lab) for each cell, record features x1 = normalised range, x2 = return
   intensity,
   and the ground-truth label m in {0, 1}
3. (lab) forward pass p_hat = sigmoid(b + w1*x1 + w2*x2); BCE loss; gradient
   descent
4. (lab) repeat to convergence; freeze (w1, w2, b); discard the ground truth
5. (field) for each new scan compute a = b + w1*x1 + w2*x2
6. (field) L_t = L_{t-1} + a - L_0; map probability = sigmoid(L_t)
  
```

4. Worked example

Exam Tip / Worked Example

Part A — one training step. Initial weights $w_1 = -0.5$, $w_2 = 1.5$, $b = 0$, learning rate $\alpha = 0.5$. Four training cells give activations and predictions:

Cell	a	$\hat{p} = \sigma(a)$	label m	BCE loss
(1,0)	1.3	0.786	0	1.542
(2,0)	-0.7	0.332	1	1.103
(3,0)	-2.7	0.063	0	0.065
(0,2)	-0.7	0.332	0	0.403
Total BCE loss				0.778

Sum of w_2 gradients = $0.943 - 0.134 - 0.050 + 0.066 = 0.825$; average gradient = $0.825/4 = 0.206$;

$$w_2^{new} = 1.5 - 0.5(0.206) = \mathbf{1.397}$$

Part B — deployment, four sweeps of the same cell. Calibrated weights: $w_1 = -5.0$ (short range $\Rightarrow$ occupied), $w_2 = +6.0$ (high intensity $\Rightarrow$ occupied), $b = -1.0$. So $a = -1 - 5x_1 + 6x_2$, and $L_t = L_{t-1} + a$.

Sweep	event	x_1	x_2	a	L_t	$\rightarrow \sigma(L_t)$
1	clear hit	0.20	0.80	+2.8	2.8	$\rightarrow 0.9427$
2	dust / forklift	0.70	0.30	-2.7	0.1	$\rightarrow 0.5250$
3	dust cleared	0.10	0.95	+4.2	4.3	$\rightarrow 0.9866$
4	final confirmation	0.10	0.90	+3.9	8.2	$\rightarrow 0.9997$

Check sweep 1 by hand: $a = -1 - 5(0.2) + 6(0.8) = -1 - 1 + 4.8 = 2.8$; $\sigma(2.8) = 1/(1 + e^{-2.8}) = 1/(1 + 0.0608) = 0.9427$. ✓

Reading: the transient obstruction in sweep 2 drags the map from 94% to 52.5% — it is heard — but two subsequent confirmations drive it to 99.97%. The filter neither ignores the disturbance nor lets it win.

5. Compared with

Comparison

Against a hand-written inverse model: strictly better wherever the sensor's physics is not analytic — glass, steam, dust, reflective metal — because it learns the actual response. Strictly worse where no labelled ground truth exists, where the environment differs from the training room, or where an auditable, explainable decision is required. It does not change the mapping algorithm at all, which is the point: it is a drop-in replacement for one term.

6. Best suited when

You can afford a supervised calibration run in a room with ground truth, and the deployment environment contains materials the analytic rangefinder model gets wrong.

7. ML / DNN / RL

This method *is* the ML integration. The escalation ladder in the deck: logistic regression (2 features, interpretable, hand-computable) $\rightarrow$ MLP (non-linear feature interactions) $\rightarrow$ 1-D CNN over the scan (spatial context, handles reflection patterns) $\rightarrow$ RNN over time (temporal patterns, distinguishes recurring from novel).

4.3 Naive-Bayes multi-sensor fusion and MAP thresholding

1. What it is

Two small ideas that always appear together. Fusion: when several sensors observe the same cell, assume they are conditionally independent given the cell state and multiply their likelihoods. MAP:

planners need a binary answer, so threshold the fused posterior at 0.5 (equivalently, at log-odds 0) to convert the probability map into an occupied/free map.

2. Canonical formulas

Conditional independence and Bayes:

$$p(z_A, z_B \mid O) = p(z_A \mid O) p(z_B \mid O), \quad p(O \mid z_A, z_B) = \eta p(z_A \mid O) p(z_B \mid O) p(O)$$

MAP decision:

$$\hat{m}_i = \arg \max_{s \in \{O, F\}} p(s \mid z_{1:t}) \iff \hat{m}_i = \text{occupied iff } l_{t,i} > 0$$

3. Algorithm

Algorithm

```
NaiveBayesFusionMAP(sensor readings z_A..z_N for cell i):
1. num_O = P(O) * PRODUCT over sensors of P(z_s | O)
2. num_F = P(F) * PRODUCT over sensors of P(z_s | F)
3. eta = 1 / (num_O + num_F)
4. P(O | z) = eta * num_O
5. MAP state = OCCUPIED if P(O | z) > 0.5 else FREE // equivalently
   log-odds > 0
6. hand the binary map to the planner; keep the probability for the next
   update
```

4. Worked example

Exam Tip / Worked Example

Two sensors on one cell. Prior $P(O) = P(F) = 0.5$.

	$P(\text{Detect})$	$P(\text{No Detect})$
Sensor A (laser), cell Occupied	0.9	0.1
Sensor A (laser), cell Free	0.1	0.9
Sensor B (ultrasonic), cell Occupied	0.8	0.2
Sensor B (ultrasonic), cell Free	0.3	0.7

Scenario 1 — both sensors detect.

$$P(z_A, z_B \mid O) = 0.9 \times 0.8 = 0.72 \Rightarrow \times 0.5 = 0.36$$

$$P(z_A, z_B \mid F) = 0.1 \times 0.3 = 0.03 \Rightarrow \times 0.5 = 0.015$$

$$P(O \mid z_A, z_B) = 0.36 / (0.36 + 0.015) = 0.36 / 0.375 = \mathbf{0.96} \Rightarrow \mathbf{MAP = Occupied}$$

Scenario 2 — sensors conflict (A detects, B does not).

$$P(z_A, z_B \mid O) = 0.9 \times 0.2 = 0.18 \Rightarrow \times 0.5 = 0.09$$

$$P(z_A, z_B \mid F) = 0.1 \times 0.7 = 0.07 \Rightarrow \times 0.5 = 0.035$$

$$P(O \mid z_A, z_B) = 0.09 / 0.125 = \mathbf{0.72} \Rightarrow \mathbf{MAP = Occupied}$$

Why the conflict resolves toward occupied: the laser's detection is 9× more likely under Occupied than Free (0.9 vs 0.1), while the ultrasonic's non-detection is only 3.5× more likely under Free (0.7 vs 0.2). $9 > 3.5$, so the laser wins. The fused belief drops from 0.96 to 0.72 — the disagreement is registered, not ignored.

5. Compared with

Comparison

Against taking the most confident sensor: fusion uses all evidence and produces a calibrated number rather than a vote. Against a full covariance model: naive Bayes assumes independence, which is wrong when two sensors share a failure mode (both blinded by the same smoke) — then it becomes overconfident. Against keeping the probability: MAP throws away the confidence, so do it *late*, at the planner boundary, never inside the filter.

6. Best suited when

Heterogeneous sensors with genuinely different failure modes (laser + ultrasonic + IR), each individually unreliable. The deck lists the practical fusion problems this must survive: sensor noise, conflicting measurements, different resolutions and fields of view, timing misalignment, and outright dropout.

7. ML / DNN / RL

Architecture / Implementation

The conditional-independence assumption is exactly what a learned fusion model removes: a small network taking all sensor channels jointly can represent correlated failures (both sensors blinded by the same smoke) that naive Bayes cannot. Input: raw channels from all sensors plus cell features. Output: a single calibrated $p(\text{occ})$. Learnt: the joint failure structure.

4.4 MAP occupancy mapping by hill climbing (forward model)

1. What it is

The forward-model answer to mapping. Instead of updating cells independently, it proposes a whole map configuration, *simulates* what the sensor would have read from that map, and scores the simulation against the actual reading. Local search then flips cell states one at a time, keeping any flip that raises the score. Because it evaluates whole configurations it does not suffer the overlap conflicts that the factorised occupancy grid produces.

2. Canonical formulas

Objective (log-likelihood of the measurement under the candidate map):

$$h(m) = \log p(z_t \mid m, x_t)$$

Under a Gaussian range error with variance σ^2 :

$$h(m) = -\frac{(z_{\text{act}} - z_{\text{sim}}(m))^2}{2\sigma^2}$$

Perfect fit $\Rightarrow h \approx 0$; poor fit $\Rightarrow h \ll 0$. The search maximises h .

3. Algorithm

Algorithm

```
HillClimbingMAPMapping(z_t, x_t, initial map m):
1. isolate the active subset S = cells crossed by the beam vector //
   line-of-sight filter
2. simulate z_sim from the current map m; score h(m) = -(z_act - z_sim)^2 /
   (2*sigma^2)
3. for each cell in S: propose flipping its state
```

4. re-simulate z_{sim} , re-score h ; ACCEPT the flip if h improves, else flip back
5. repeat until no single flip improves h // local optimum
6. return the maximising configuration

4. Worked example

Exam Tip / Worked Example

Robot pose $(0, 1)$ facing 0° . Measured range $z_{act} = 3.0$. Likelihood $z \sim \mathcal{N}(d, 0.25)$, so $2\sigma^2 = 0.5$ and $h(m) = -(3 - z_{sim})^2/0.5$.

Subsampling first. The grid is 4×4 , so there are $2^{16} = 65,536$ possible maps. Only the three cells on the beam's line of sight can change z_{sim} , so the active subset is $S = \{(1, 1), (2, 1), (3, 1)\}$ and the search space collapses to $2^3 = 8$. *This reduction is the method.*

Config	occupied cells	z_{sim}	$h(m)$	decision
m_0	none (all free)	4.0 (max range)	-2.0	baseline
m_1	flip (1,1)	1.0	-8.0	reject, worse than -2.0
m_2	flip (2,1)	2.0	-2.0	reject, no gain
m_3	flip (3,1)	3.0	0.0	accept

Check m_1 by hand: $-(3 - 1)^2/0.5 = -4/0.5 = -8.0$. ✓

Termination: $S^* = \{(1, 1):0, (2, 1):0, (3, 1):1\}$ — exactly one obstacle, at range 3, which is what the sensor said. Note that m_2 scored equal to the baseline, not better; strict hill climbing rejects ties, which is what keeps the map sparse.

5. Compared with

Comparison

Against the log-odds occupancy grid: resolves the overlap conflict that the independence assumption creates, because it explains the whole scan with one configuration. Costs vastly more — a combinatorial search per scan versus one addition per cell — and delivers a *local* optimum with no confidence attached. In practice the grid is used online and this is used to reason about what the grid gets wrong.

Limitation / Warning

Hill climbing has no destination and no heuristic in the A* sense — the objective function *is* the heuristic. It gets stuck in local maxima, it has no guarantee of finding the global MAP map, and without the line-of-sight subsampling the search space is $2^{(\text{number of cells})}$, which is hopeless for a real map with 250,000 cells.

6. Best suited when

Offline map refinement, small regions, or explaining a specific inconsistency in a grid map built online. Also the right conceptual answer when an exam asks why the occupancy grid's independence assumption is wrong and what fixes it.

7. ML / DNN / RL

Architecture / Implementation

Simulator as data generator — the deck’s framing. Running the forward simulation on 250,000 cells every millisecond would melt the CPU; so simulate *once* per area, save the resulting parameters, and afterwards read from memory in under a microsecond. The learned inverse model of the previous method is precisely the compressed, cached form of this forward simulation. RL enters through the search step: a policy that chooses which cells to flip next (rather than sweeping the subset) reaches the optimum in fewer evaluations.

5 Localization

The four-axis taxonomy — name the problem before choosing the method

Axis	One end	Other end
Environment	Static — walls and landmarks fixed	Dynamic — people, vehicles, the attacker pivots
Agents	Single — one robot, one belief	Multiple — shared map, joint belief grows exponentially
Knowledge	Local (tracking) — tight unimodal belief, small corrections	Global (kidnapped) — belief starts uniform, must converge from scratch
Motion control	Passive — read whatever u the task produces	Active — choose u to reduce belief entropy

A Roomba in a quiet room is STATIC · SINGLE · LOCAL · PASSIVE, and the textbook grid filter is enough. The EC3 rescue robot “may unknowingly be picked up and moved while still powered on” — that is GLOBAL, and it is the clause that decides question 4(c).

5.1 Frame transformation (global map frame $\leftrightarrow$ local robot frame)

1. What it is

Not a filter, but the arithmetic every filter depends on. The map frame is fixed and knows nothing about the robot; the robot frame has its origin at the robot with x' along its heading. Converting between them is always translate-then-rotate one way and rotate-then-translate the other, and getting the order or the sign of θ wrong is the single most common source of a wrong localization answer.

2. Canonical formulas

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}, \quad R(-\theta) = R(\theta)^T = \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix}$$

Map $\rightarrow$ robot (translate first, then rotate by $-\theta$):

$$\begin{pmatrix} x_r \\ y_r \end{pmatrix} = R(-\theta) \begin{pmatrix} X - x \\ Y - y \end{pmatrix}$$

Robot $\rightarrow$ map (rotate by $+\theta$ first, then translate):

$$\begin{pmatrix} X \\ Y \end{pmatrix} = R(\theta) \begin{pmatrix} x_r \\ y_r \end{pmatrix} + \begin{pmatrix} x \\ y \end{pmatrix}$$

3. Algorithm

Algorithm

```

MapToRobot(landmark (X,Y), robot pose (x,y,theta)):
1.  translate:  dx = X - x; dy = Y - y
2.  rotate by -theta:  x_r = cos(theta)*dx + sin(theta)*dy
3.                      y_r = -sin(theta)*dx + cos(theta)*dy
RobotToMap(point (x_r,y_r), robot pose (x,y,theta)):
4.  rotate by +theta:  X' = cos(theta)*x_r - sin(theta)*y_r
5.                      Y' = sin(theta)*x_r + cos(theta)*y_r
6.  translate:  X = X' + x; Y = Y' + y    // note: rotate BEFORE translate
here

```

4. Worked example**Exam Tip / Worked Example**

Robot pose (2,3) with $\theta = 45^\circ = \pi/4$. Landmark at (4,5) in the map frame. Where is the landmark in the robot's own frame?

Step 1 — translate. $(X - x, Y - y) = (4 - 2, 5 - 3) = (2, 2)$.

Step 2 — rotate by -45° . $\cos 45^\circ = \sin 45^\circ = 0.7071$.

$$x_r = 0.7071(2) + 0.7071(2) = \mathbf{2.8284}, \quad y_r = -0.7071(2) + 0.7071(2) = \mathbf{0}$$

So in the robot frame the landmark is at (2.83, 0) — **straight ahead**, which is the whole point: the robot is at (2,3) facing 45° and the landmark is at (4,5), exactly along its heading. Range $= \sqrt{2^2 + 2^2} = 2.8284$, bearing = 0.

Round trip, to check. Rotate (2.8284, 0) by $+45^\circ$: $X' = 0.7071(2.8284) - 0.7071(0) = 2$, $Y' = 0.7071(2.8284) + 0 = 2$. Translate: $(2 + 2, 2 + 3) = \mathbf{(4, 5)}$. ✓

Exam habit: always sanity-check that a landmark straight ahead has $y_r = 0$, and that the range is preserved by the rotation. Both checks take five seconds and catch a sign error.

5. Compared with**Comparison**

There is no alternative — but there is a wrong version. Translating *after* rotating on the map→robot direction (or rotating by $+\theta$ instead of $-\theta$) gives an answer that is right only when $\theta = 0$, which is exactly the case most people test with. The likelihood-field model depends on this transform: without it, sensor readings move with the robot and can never be “stuck” to fixed obstacles.

6. Best suited when

Every time a measurement in the robot's frame must be compared against a map, and every time a planned waypoint in the map frame must be turned into a command.

7. ML / DNN / RL

No learning is needed or wanted here — the transform is exact. The learning happens immediately downstream: once endpoints are in the map frame, a network can score them (likelihood field), classify them (PointNet++), or match them (SuperPoint).

5.2 Grid-based localization (histogram filter)

1. What it is

The discrete Bayes filter applied to pose. The map is chopped into cells, belief is one number per cell, and each cycle runs a predict step through a motion transition table and an update step through a measurement likelihood table. Because it keeps mass everywhere, it solves global localization and can recover from kidnapping — which is exactly what a Kalman filter cannot do.

2. Canonical formulas

Identical to the discrete Bayes filter, with x ranging over cells:

$$\overline{bel}(x_t^k) = \sum_i p(x_t^k | u_t, x_{t-1}^i) bel(x_{t-1}^i), \quad bel(x_t^k) = \eta p(z_t | x_t^k) \overline{bel}(x_t^k)$$

Motion noise is often injected by inflation: add a small probability proportional to movement/cellSize to neighbouring cells.

3. Algorithm

Algorithm

```
GridLocalization({bel(x_{t-1})}, u_t, z_t, map m):
1. for every cell k: bel_bar(k) = SUM over i of P(x_t=k | u_t, x_{t-1}=i) * bel(i)
2. for every cell k: bel(k) = P(z_t | x_t = k, m) * bel_bar(k)
3. eta = 1 / SUM over k of bel(k); bel = eta * bel
4. (optimisation) skip cells whose belief is below a threshold - selective updating
5. (optimisation) pre-cache P(z|x) per cell, subsample beams, delay motion updates
6. return bel
```

4. Worked example

Exam Tip / Worked Example

Four candidate positions $x \in \{5, 10, 15, 20\}$, uniform prior 0.25 each.

Motion table $p(x_t | u_t, x_{t-1})$ (rows = from, columns = to):

from \ to	5	10	15	20
5	0.2	0.6	0.2	0.0
10	0.3	0.1	0.6	0.1
15	0.1	0.3	0.1	0.5
20	0.0	0.1	0.3	0.6

Predict step — each column, weighted by the uniform prior 0.25:

$$\overline{bel}(5) = 0.25(0.2 + 0.3 + 0.1 + 0.0) = 0.25(0.6) = \mathbf{0.150}$$

$$\overline{bel}(10) = 0.25(0.6 + 0.1 + 0.3 + 0.1) = 0.25(1.1) = \mathbf{0.275}$$

$$\overline{bel}(15) = 0.25(0.2 + 0.6 + 0.1 + 0.3) = 0.25(1.2) = \mathbf{0.300}$$

$$\overline{bel}(20) = 0.25(0.0 + 0.1 + 0.5 + 0.6) = 0.25(1.2) = \mathbf{0.300}$$

(Check: $0.150 + 0.275 + 0.300 + 0.300 = 1.000$. ✓ Always verify this before continuing.)

Update step with $z_t = 15$. Likelihoods $p(Z=15 | x)$: 0.1, 0.2, 0.4, 0.1.

x	5	10	15	20
$\overline{bel}$	0.150	0.275	0.300	0.300
$p(z \mid x)$	0.1	0.2	0.4	0.1
product	0.015	0.055	0.120	0.030
$bel (\eta = 1/0.22)$	0.068	0.250	0.545	0.136

η denominator = $0.015 + 0.055 + 0.120 + 0.030 = 0.220$.

Reading: prediction left $x=15$ and $x=20$ tied at 0.30; one measurement broke the tie decisively (0.545 vs 0.136). Note that mass never left $x=5$ entirely — that residual 6.8% is what lets the filter recover if the robot turns out to be somewhere unexpected. A Kalman filter would have discarded it.

5. Compared with

Comparison

Against	Grid filter wins because	Grid filter loses because
Kalman / EKF	multimodal belief, no linearity assumption, survives kidnapping	cost is $O(\text{cells})$ per step and scales terribly with resolution and dimension
MCL (particle filter)	deterministic and reproducible; resolution is uniform and known	particles concentrate compute where the probability actually is; the grid spends effort on empty cells

The deck's own trade-off framing: *accuracy vs resource intensity*. Fine-grained fixed grids buy precision and are rescued by pre-caching, sensor subsampling, delayed motion updates, selective updating and heuristics. Coarse-grained dynamic grids buy speed and are rescued by topological representation and by *inflating* both motion and measurement noise so the coarse cells stay honest.

6. Best suited when

Global localization in a modest, mostly-2-D map where you want a guaranteed-resolution answer and can afford the sweep. Also the correct pedagogical answer whenever the exam asks for a localization update “step by step” with a table.

7. ML / DNN / RL

Architecture / Implementation

Four concrete entry points, straight from the deck:

- Fusing many noisy sensors.** Train a net mapping (cell, raw packet/LiDAR/WiFi) to a likelihood, and plug that into the histogram update. Use case: multi-sensor mobile robot.
- Camera streams as measurements.** A frame is too high-dimensional for an analytical likelihood; a CNN outputs a heatmap over cells, and *that heatmap is $p(z_t \mid x_t)$* . Use case: visual localization with no LiDAR.
- Action recommendation (active localization).** Computing information gain analytically is slow; RL learns $\pi(u \mid bel)$ directly. Use case: kidnapped-robot recovery.
- Learning the motion noise.** Instead of hand-picking 20% or 50% inflation, train a model to predict per-cell noise from odometry against ground truth. Use case: AMR on slippery surfaces.

5.3 Monte Carlo Localization (particle filter)

1. What it is

Represents the belief as a set of M weighted pose samples instead of a table or a Gaussian. Each cycle pushes every particle through the motion model, weights it by how well its predicted measurement matches the actual one, and resamples proportionally to weight — so particles migrate toward regions of high posterior probability. It is the non-parametric filter of the course: multimodal, no linearity assumption, and compute spent only where probability lives.

2. Canonical formulas

Importance weight for particle m :

$$w_t^{[m]} = p(z_t | x_t^{[m]}, m)$$

which, for a range reading with Gaussian noise, is the familiar

$$w^{[m]} = \exp\left(-\frac{(z_{meas} - z_{exp}^{[m]})^2}{2\sigma^2}\right)$$

Normalised belief $bel(x_t^{[m]}) = w_t^{[m]} / \sum_j w_t^{[j]}$. Resampling draws M particles with replacement with probability $bel(x_t^{[m]})$ (roulette wheel — the same mechanism as GA parent selection), after which all weights are reset to $1/M$.

3. Algorithm

Algorithm

```
MCL(particle set X_{t-1}, u_t, z_t, map m):
1. Initialize M particles with uniform belief (only at t = 0)
2. Predict: move every particle through the motion model with sampled noise
3. Correct: for each particle compute the expected reading z_exp and compare
with z_meas
4. Preprocess: optionally reject surprising measurements outright
5. Weigh: w = exp(-(z_meas - z_exp)^2 / (2*sigma^2)); normalise to get bel
6. Resample M particles by roulette wheel on bel; reset all weights to 1/M;
repeat
```

4. Worked example

Exam Tip / Worked Example

$M = 4$ particles, 2-D, 5×5 arena, landmark at $L = (4, 4)$, motion = 1 m east, sensor noise $\sigma^2 = 0.5$, actual reading $z_{meas} = 5$ m. Prior $bel(x_0) = 0.25$ each.

	P_1	P_2	P_3	P_4
pose x_0	(1,1,0)	(2,2,0)	(3,1,0)	(1,3,0)
pose x_1 after +1 east	(2,1,0)	(3,2,0)	(4,1,0)	(2,3,0)
$z_{exp} = \ L - x_1\ $	3.6056	2.2361	3.0000	2.2361
$w = e^{-(5-z_{exp})^2/1}$	0.14306	0.00048	0.01832	0.00048
$bel(x_1)$	0.8812	0.0030	0.1128	0.0030

Check P_1 by hand: $z_{exp} = \sqrt{(4-2)^2 + (4-1)^2} = \sqrt{4+9} = \sqrt{13} = 3.6056$; $(5 - 3.6056)^2 = 1.9444$; $w = e^{-1.9444/1} = 0.14306$. Sum of weights = $0.14306 + 0.00048 + 0.01832 + 0.00048 = 0.16234$; $bel(P_1) = 0.14306/0.16234 = 0.8812$. ✓

Resample by roulette wheel: P_1 takes 88% of the wheel and P_3 takes 11%, so a typical draw returns $\{(2, 1), (4, 1), (4, 1), (2, 1)\}$ — P_2 and P_4 die out. Belief resets to 0.25 each.

Errata note: the deck prints $bel = (0.8805, 0.0013, 0.1132, 0.0031)$, which comes from rounding z_{exp} to 3.6 before weighting; the third entry (0.0013) is also a typo — P_2 and P_4 are identical by symmetry and must carry the same belief, and the four printed values sum to 0.998, not 1. Values above are exact. See Section 10.

5. Compared with

Comparison

Against the grid filter: adaptive resolution, so it handles bigger spaces at the same compute, but it is stochastic (two runs differ) and can suffer particle deprivation. Against EKF: multimodal and non-linear, but M times more expensive and with no closed-form covariance. Against *augmented* MCL: plain MCL cannot recover from kidnapping, because after convergence every particle sits in the wrong place and resampling only ever draws from the existing set — there is no mechanism to put a particle where the robot actually is.

6. Best suited when

Global localization in a large or non-convex map, multimodal belief from symmetric geometry, and a sensor model too awkward for a Gaussian — with the caveat that the robot is not going to be teleported.

7. ML / DNN / RL

See the sampling-based motion model: a learned proposal $q(x_t | x_{t-1}, z_t, u_t)$ places particles where the posterior mass is, and a differentiable particle filter trains proposal and sensor model jointly. NVIDIA's differentiable PF achieves equal accuracy with 100× fewer particles.

5.4 Augmented MCL

1. What it is

Plain MCL with a kidnapping detector bolted on. It tracks two running averages of the mean particle weight — a fast one and a slow one — and compares them. If the short-term average collapses relative to the long-term average, the particles have stopped explaining the sensor data, which is the signature of the robot having been moved. The filter responds by injecting random particles across the map, restoring the diversity that resampling destroyed.

2. Canonical formulas

$$w_{avg} = \frac{1}{M} \sum_{m=1}^M w_t^{[m]}$$

$$w_{fast} \leftarrow w_{fast} + \alpha_{fast} (w_{avg} - w_{fast}), \quad w_{slow} \leftarrow w_{slow} + \alpha_{slow} (w_{avg} - w_{slow})$$

with $\alpha_{fast} \gg \alpha_{slow}$ (the deck uses 0.5 and 0.1). Random-injection probability per resampled particle:

$$p_{random} = \max\left(0, 1 - \frac{w_{fast}}{w_{slow}}\right)$$

3. Algorithm

Algorithm

```

AugmentedMCL( $X_{t-1}$ ,  $u_t$ ,  $z_t$ ,  $m$ ,  $\alpha_{fast}$ ,  $\alpha_{slow}$ ):
1. run MCL steps 2-5 (predict, correct, weigh) exactly as before
2.  $w_{avg}$  = mean of the  $M$  raw weights
3.  $w_{fast} = (1 - \alpha_{fast})w_{fast} + \alpha_{fast}w_{avg}$ 
4.  $w_{slow} = (1 - \alpha_{slow})w_{slow} + \alpha_{slow}w_{avg}$ 
5.  $p_{random} = \max(0, 1 - w_{fast} / w_{slow})$ 
6. resample: with prob  $p_{random}$  draw a uniformly random pose, else
   roulette-wheel as usual

```

4. Worked example**Exam Tip / Worked Example**

Continue the MCL example above. Raw weights were 0.14306, 0.00048, 0.01832, 0.00048. Take $\alpha_{fast} = 0.5$, $\alpha_{slow} = 0.1$, and stored histories $w_{fast} = 0.5$, $w_{slow} = 0.9$ (values chosen so the mechanism is visible).

Mean weight:

$$w_{avg} = \frac{0.14306 + 0.00048 + 0.01832 + 0.00048}{4} = \frac{0.16234}{4} = \mathbf{0.04058}$$

Update the two averages:

$$w_{fast} = (1 - 0.5)(0.5) + 0.5(0.04058) = 0.25 + 0.02029 = \mathbf{0.2703}$$

$$w_{slow} = (1 - 0.1)(0.9) + 0.1(0.04058) = 0.81 + 0.00406 = \mathbf{0.8141}$$

Injection probability:

$$\frac{w_{fast}}{w_{slow}} = \frac{0.2703}{0.8141} = 0.3320 \quad \Rightarrow \quad p_{random} = \max(0, 1 - 0.3320) = \mathbf{0.668}$$

Decision: roughly 33% of the particles are informative, so retain them by roulette wheel; the remaining $\approx 67\%$ of M are drawn uniformly at random from the map. With $M = 4$ that is one survivor and three fresh random poses.

Why it works: w_{fast} tracks the current scan; w_{slow} remembers what a healthy match used to look like. Their ratio is a dimensionless health score — it does not care about the absolute scale of the weights, only that the present is worse than the past. If nothing has changed, $w_{fast} \approx w_{slow}$, the ratio is ≈ 1 , and p_{random} clamps to 0: augmented MCL is then exactly plain MCL, costing nothing.

5. Compared with**Comparison**

Strictly dominates plain MCL: identical behaviour when nothing goes wrong, plus recovery when something does. The price is two extra scalars and the risk of injecting particles spuriously when the sensor is merely noisy (tune α_{fast} down if this happens). Against a grid filter, which also survives kidnapping by keeping mass everywhere: augmented MCL gets the same robustness without paying for cells that hold no probability.

6. Best suited when

Any deployment where the robot can be displaced without knowing it — carried, lifted onto a charger,

pushed, or (in the software analogue) migrated to a different subnet.

EC3 Q4(c) — the planted clause

“The robot *may unknowingly be picked up and moved by a rescue worker while still powered on.*” That is the textbook definition of the kidnapped-robot problem, and it is the entire question. Plain MCL converges its particles onto the pre-kidnap pose; after the move, every particle is wrong, all weights collapse together, and roulette-wheel resampling can only redistribute among particles that are already wrong — the filter is confidently lost with no route back. Augmented MCL notices the w_{fast}/w_{slow} collapse and injects random particles, some of which land near the kiosk landmark and re-anchor the belief. **Answer: Augmented MCL.**

7. ML / DNN / RL

Architecture / Implementation

The $\alpha_{fast}/\alpha_{slow}$ pair is hand-tuned and environment-specific — the obvious learning target. More substantially, this is where **RL for active localization** pays off: once p_{random} spikes, the robot should not keep executing its task blindly; a policy $\pi(u | bel)$ rewarded by belief-entropy reduction will deliberately drive toward a disambiguating landmark (in the EC3 scenario, the intact information kiosk). That is the deck’s “active localization” quadrant and its stated major entry point for RL.

5.5 Mixture (dual) MCL

1. What it is

Swaps the roles of the two models for a fraction ϕ of the particles. Normal MCL samples poses from the *motion* model and weights them by the *sensor* model. Dual MCL does the reverse: it samples poses directly from the sensor model — “where could I be, given this reading?” — and weights them by the motion model. Mixture MCL runs a proportion ϕ of dual particles alongside $1 - \phi$ normal ones, which fixes the case where the motion model is so noisy that it never proposes a particle near the truth.

2. Canonical formulas

Mixture proposal:

$$q(x_t) = (1 - \phi) \underbrace{p(x_t | u_t, x_{t-1})}_{\text{normal: motion proposal}} + \phi \underbrace{p(x_t | z_t)}_{\text{dual: sensor proposal}}$$

Dual particles carry the motion-model likelihood as their weight rather than the sensor likelihood, since the sensor information has already been consumed in generating them.

3. Algorithm

Algorithm

```
MixtureMCL(X_{t-1}, u_t, z_t, m, phi):
1. draw (1 - phi)*M particles the normal way: propagate through the motion model
2. draw phi*M particles from the sensor model p(x | z_t) - poses consistent with the reading
3. weigh the normal particles by the sensor likelihood p(z_t | x)
4. weigh the dual particles by the motion likelihood p(x | u_t, x_{t-1})
```

```

5. normalise across the whole set to get bel
6. resample phi*M from the dual pool and (1 - phi)*M from the normal pool;
repeat

```

4. Worked example

Exam Tip / Worked Example

Same arena, $L = (4, 4)$, $z_{meas} = 5$, $\sigma^2 = 0.5$, $\phi = 0.25$ — so one of the four particles (P_1) is drawn from the sensor model instead of the motion model.

P_1 is placed at $(0.5, 0.5, 0)$, a pose consistent with a 5 m reading to L :

$$z_{exp} = \sqrt{(4 - 0.5)^2 + (4 - 0.5)^2} = \sqrt{12.25 + 12.25} = \sqrt{24.5} = \mathbf{4.9497}$$

$$w_1 = \exp\left(-\frac{(5 - 4.9497)^2}{2(0.5)}\right) = \exp(-0.00253) = \mathbf{0.99748}$$

P_2, P_3, P_4 keep their motion-derived poses and weights 0.00048, 0.01832, 0.00048. Sum = $0.99748 + 0.01928 = 1.01676$.

$$bel = (\mathbf{0.9810}, 0.00047, 0.01802, 0.00047)$$

Note the failure the deck flags. P_1 now holds 98% of the belief. If you resample with a plain roulette wheel, all four next-generation particles come from P_1 — the swarm collapses onto a single hypothesis and diversity is gone. The fix is to resample *by pool*: draw $\phi M = 1$ particle from the dual pool and $(1 - \phi)M = 3$ from the motion pool, rather than letting one dominant weight take the whole wheel. The deck also lists the alternative fix: combine with augmented MCL, so the w_{fast}/w_{slow} ratio governs how much exploration is forced.

5. Compared with

Comparison

Fixes the one thing neither plain nor augmented MCL can: a motion model so noisy or so wrong that its proposals never land near the truth. Costs the ability to sample from $p(x | z)$, which requires an invertible sensor model — easy for range-to-a-known-landmark, hard for a full LiDAR scan. Augmented MCL fixes *kidnapping*; mixture MCL fixes a *bad motion model*. They are complementary, and the deck combines them.

6. Best suited when

Odometry is badly degraded (rubble, slip, a dragged robot) but the sensor is informative and easy to invert — for example a distinctive landmark that pins the range directly.

7. ML / DNN / RL

Architecture / Implementation

The deck's own conclusion on this slide: “use AI/ML to determine the balance between exploration and exploitation”. That is, learn ϕ instead of fixing it. A bandit or RL controller observing recent weight statistics chooses per-step how many dual particles to draw — more when odometry is failing, none when it is healthy. The sensor-model sampler itself is also a learning target: a conditional generative model $x \sim p(x | z)$ trained on (pose, scan) pairs

replaces the analytic inversion and extends dual MCL to full LiDAR scans.

6 SLAM

Two axes classify every SLAM method in this course

What is stored as the map? Grid-based (dense occupancy) or feature-based (landmarks). Everything below is feature-based — the course uses feature maps for the whole SLAM module.

How is uncertainty represented? Histogram, Gaussian (EKF-SLAM, EIF, SEIF), or particle set (FastSLAM).

And one more: *online* SLAM estimates $p(x_t, m \mid z_{1:t}, u_{1:t})$ — the current pose only. *Full* SLAM estimates $p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$ — the whole trajectory. EKF and SEIF are online; EIF and GraphSLAM are full/batch.

Data association is the thing that actually breaks SLAM

“Which landmark in the map does this new observation correspond to?” Four methods, weakest first: **nearest neighbour** (closest predicted landmark — simple, fast); **Mahalanobis distance** (accounts for uncertainty — the EKF standard); **joint compatibility** (checks consistency across multiple observations — most robust); **visual descriptors** (ORB/SIFT/BRIEF — visual SLAM). Gate with $d^2 = (z - \hat{z})^T S^{-1} (z - \hat{z}) < \text{threshold}$, and pick the landmark with the smallest d^2 . A wrong association is not a small error — it welds two different places together and corrupts the map permanently.

6.1 EKF-SLAM

1. What it is

Puts the robot pose and every landmark into one joint Gaussian state vector and runs an extended Kalman filter over it. The key property is that the covariance matrix is *dense*: observing one landmark updates the estimate of every other landmark, because they are all correlated through the robot’s path. That is why re-observing a known landmark shrinks the uncertainty of the whole map at once — and also why the cost grows quadratically with the number of landmarks.

2. Canonical formulas

Joint state $y = [x_r, m_1, m_2, \dots]^T$. **Prediction:**

$$\mu^- = g(u_t, \mu_{t-1}), \quad \Sigma^- = G_t \Sigma_{t-1} G_t^T + Q_t$$

Correction:

$$S = H \Sigma^- H^T + R, \quad K = \Sigma^- H^T S^{-1}, \quad \mu = \mu^- + K(z - h(\mu^-)), \quad \Sigma = (I - KH) \Sigma^-$$

S is the innovation covariance — prediction uncertainty plus sensor uncertainty. K is the Kalman gain: *how much to trust the measurement versus the prediction*. Noisy sensor (R large) $\Rightarrow$ small K , trust the prediction. Uncertain prediction (Σ^- large) $\Rightarrow$ large K , trust the measurement.

3. Algorithm

Algorithm

```

EKF-SLAM(mu, Sigma, u_t, z_t):
1. Initialization: mu = [robot pose, landmark means], Sigma = block-diagonal
2. Prediction: apply the motion model to get mu_minus; Sigma_minus =
   G*Sigma*G^T + Q
3. Correction (a): apply the measurement model h(mu_minus) for each landmark
4. Correction (b): data association - pick the landmark with smallest
   Mahalanobis d^2
5. Correction (c): S = H*Sigma_minus*H^T + R; K = Sigma_minus*H^T*S^-1
6. mu = mu_minus + K*E; Sigma = (I - K*H)*Sigma_minus; repeat

```

4. Worked example — 1-D, one robot and two landmarks**Exam Tip / Worked Example**

Initial state. $\mu_0 = [0, 4, 9]^T$ (robot at 0, L_1 at 4, L_2 at 9), $\Sigma_0 = \text{diag}(1, 4, 4)$.

Step 2 — Prediction. Control $u = 2$, so $x'_r = x_r + u$:

$$\mu^- = [2, 4, 9]^T$$

Motion noise 0.5 on the robot only, $Q = \text{diag}(0.5, 0, 0)$:

$$\Sigma^- = \Sigma_0 + Q = \text{diag}(1.5, 4, 4)$$

(The landmarks did not move, so only the robot's variance grew.)

Step 3a — Measurement model. $h(x) = m_i - x_r$, measured $z_m = 2.5$, $R = 0.5$.

$$L_1: h_1 = 4 - 2 = 2 \Rightarrow E_1 = 2.5 - 2 = +0.5$$

$$L_2: h_2 = 9 - 2 = 7 \Rightarrow E_2 = 2.5 - 7 = -4.5$$

Jacobians: $H_{L_1} = [-1, 1, 0]$, $H_{L_2} = [-1, 0, 1]$.

Step 3b — Data association. $S = H\Sigma^-H^T + R = 1.5 + 4 + 0.5 = 6$ for both (symmetric setup), so $S^{-1} = 0.1667$:

$$d_{L_1}^2 = \frac{0.5^2}{6} = 0.0417, \quad d_{L_2}^2 = \frac{(-4.5)^2}{6} = \frac{20.25}{6} = 3.375$$

$0.0417 \ll 3.375$, so the measurement is associated with L_1 . (The deck prints $d_{L_1}^2 = 0.0147$; the correct value is $0.25/6 = 0.0417$. The conclusion is unchanged. See Section 10.)

Step 3c — Kalman gain.

$$\Sigma^- H_{L_1}^T = \text{diag}(1.5, 4, 4) \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -1.5 \\ 4 \\ 0 \end{pmatrix} \Rightarrow K = \frac{1}{6} \begin{pmatrix} -1.5 \\ 4 \\ 0 \end{pmatrix} = \begin{pmatrix} -0.25 \\ 0.667 \\ 0 \end{pmatrix}$$

Update the mean.

$$\mu = \mu^- + K E_1 = \begin{pmatrix} 2 \\ 4 \\ 9 \end{pmatrix} + 0.5 \begin{pmatrix} -0.25 \\ 0.667 \\ 0 \end{pmatrix} = \begin{pmatrix} 1.875 \\ 4.333 \\ 9 \end{pmatrix}$$

Update the covariance.

$$\Sigma = (I - K H_{L_1}) \Sigma^- = \begin{pmatrix} 1.125 & 1.000 & 0 \\ 1.000 & 1.333 & 0 \\ 0 & 0 & 4 \end{pmatrix}$$

Read the three things that matter. (i) The robot moved *backwards* to 1.875 and L_1 moved *forwards* to 4.333 — the filter split the 0.5 m innovation between them, in proportion to their uncertainties. (ii) The off-diagonal 1.000 appeared where there was a zero: robot and L_1 are now **correlated**, and this is the defining feature of EKF-SLAM. (iii) L_2 is untouched at variance 4 — it was never observed and had no prior correlation. On the *next* observation of L_1 , that new correlation will propagate information to L_2 as well.

5. Compared with

Comparison

	EKF-SLAM	GraphSLAM	FastSLAM
Approach	filtering, online, recursive	optimisation, batch or pose-graph	particle filter + per-particle EKFs
Map	dense covariance over all landmarks	sparse graph of poses and constraints	each particle carries its own landmark set
Scalability	poor — $O(N^2)$ in landmarks	high — sparse structure	better — each particle handles a subset
Loop closure	implicit, weak — only through the innovation step	explicit and strong — poses re-optimised globally	implicit, via particles
Drift correction	limited	global	moderate to strong
Data association	one global decision; a mistake is unrecoverable	can be revisited	per particle — wrong guesses die out

Limitation / Warning

Advantages: maintains full correlations between robot and landmarks; quite accurate for small maps. **Disadvantages:** computational cost grows quadratically with landmark count; inverting $S = H\Sigma H^T + R$ is expensive for large states; sensitive to linearisation error (it is an EKF); not viable for very large environments; and a single mis-association is permanent.

6. Best suited when

Small to medium feature maps, a Gaussian and roughly unimodal belief, mildly non-linear dynamics, and reliable data association — the classic case being a modest indoor space with a handful of well-separated, distinctive landmarks.

7. ML / DNN / RL

Architecture / Implementation

Traditional: EKF linearises via hand-derived Jacobians with a hand-set Q — brittle for fast or highly non-linear dynamics.

ML integration (Neural Kalman Filter): a deep network replaces $f(x, u)$; Jacobians come from automatic differentiation and Q , R are learnt jointly end-to-end. **Input:** x_{t-1} , u_t , z_t .

Output: Gaussian belief $\mathcal{N}(\mu_t, \Sigma_t)$. **Learnt:** non-linear transition dynamics and the process noise inferred from observed residuals. **Deck example:** Deep Kalman Filter (Uber AI Labs, 2017); learned EKF Jacobians for racing-drone localization at 200 Hz.

Data association — ML: learn the Mahalanobis gate from noise logs (MLE); **DNN**

(YOLO/SSD) extracts semantic landmark identities so “Kitchen Door” and “Hallway Door” stop being geometrically identical — the direct fix for perceptual aliasing.

6.2 EIF-SLAM (Extended Information Form)

1. What it is

The same Gaussian, written the other way round. Instead of carrying the mean μ and covariance Σ , it carries the information matrix $\Omega = \Sigma^{-1}$ and the information vector $\xi = \Omega\mu$. The payoff is that measurements are *additive* in information space — integrating an observation is a matrix addition, not a Kalman-gain computation — and the information matrix of a large map is largely zeros, since most landmark pairs are never seen together. The cost is that recovering μ requires solving $\Omega\mu = \xi$.

2. Canonical formulas

$$\Omega = \Sigma^{-1}, \quad \xi = \Omega\mu, \quad \mu = \Omega^{-1}\xi$$

Measurement update — pure addition:

$$\Omega = \Omega^- + H^T R^{-1} H, \quad \xi = \xi^- + H^T R^{-1} (z - h(\mu^-) + H\mu^-)$$

Motion update — the awkward one: $\Omega^- = (G_t \Omega_{t-1}^{-1} G_t^T + Q_t)^{-1}$. The deck’s own note: EIF cannot inject motion noise directly in information space, so it temporarily inverts into covariance space, does the EKF step, and inverts back.

3. Algorithm

Algorithm

```
EIF-SLAM(Omega, xi, u_t, z_t):
1. Initialization:  Omega = Sigma^-1; xi = Omega * mu
2. Prediction:  invert to Sigma, add motion noise Q as in EKF, invert back to Omega_minus
3.    xi_minus = Omega_minus * mu_minus
4. Correction:  Omega = Omega_minus + H^T * R^-1 * H    // additive, no gain needed
5.    xi = xi_minus + H^T * R^-1 * (z - h(mu_minus) + H*mu_minus)
6. State recovery only when queried:  solve Omega * mu = xi
```

4. Worked example — identical numbers to the EKF above

Exam Tip / Worked Example

Same problem: $\mu_0 = [0, 4, 9]$, $\Sigma_0 = \text{diag}(1, 4, 4)$, $u = 2$, noise 0.5, $z_m = 2.5$, $R = 0.5$, $H = [-1, 1, 0]$.

Initialise in information form.

$$\Omega_0 = \Sigma_0^{-1} = \text{diag}(1, 0.25, 0.25), \quad \xi_0 = \Omega_0 \mu_0 = [0, \mathbf{1}, \mathbf{2.25}]^T$$

Predict. $\Sigma^- = \text{diag}(1.5, 4, 4)$ as before, so

$$\Omega^- = \text{diag}(\mathbf{0.667}, 0.25, 0.25), \quad \xi^- = \Omega^- \mu^- = [0.667(2), 0.25(4), 0.25(9)] = [\mathbf{1.333}, 1, 2.25]$$

Note $0.667 < 1$: motion *destroyed* information about the robot, exactly as it inflated the variance in the EKF.

Correct. $R^{-1} = 2$ and $H^T H = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$, so

$$\Omega = \Omega^- + 2H^T H = \begin{pmatrix} 2.667 & -2.0 & 0 \\ -2.0 & 2.25 & 0 \\ 0 & 0 & 0.25 \end{pmatrix}$$

The off-diagonal -2.0 is the *link* created between robot and L_1 by the observation. For ξ : $z - h(\mu^-) = 2.5 - 2 = 0.5$ and $H\mu^- = -1(2) + 1(4) = 2$, so the bracket is $0.5 + 2 = 2.5$ and

$$\xi = \xi^- + [-1, 1, 0]^T (2)(2.5) = [1.333 - 5, 1 + 5, 2.25] = [-3.667, 6.0, 2.25]^T$$

Recover the state. L_2 is decoupled, so read it straight off: $\mu_{m_2} = 2.25/0.25 = 9$. For the 2×2 block, solve $\begin{pmatrix} 2.667 & -2.0 \\ -2.0 & 2.25 \end{pmatrix} \begin{pmatrix} \mu_r \\ \mu_{m_1} \end{pmatrix} = \begin{pmatrix} -3.667 \\ 6.0 \end{pmatrix}$. Determinant = $2.667(2.25) - 4 = 6 - 4 = 2$, so

$$\mu_r = \frac{(-3.667)(2.25) - (-2)(6)}{2} = \frac{-8.25 + 12}{2} = 1.875$$

$$\mu_{m_1} = \frac{2.667(6) - (-2)(-3.667)}{2} = \frac{16 - 7.333}{2} = 4.333$$

$\mu = [1.875, 4.333, 9]$ — **exactly the EKF answer.** That is the point of the example: same Gaussian, two parameterisations, one truth.

5. Compared with

Comparison

Pros (deck): information matrices are often sparse (many zeros), so storage and computation are more efficient; useful in large-scale SLAM where landmarks are sparsely correlated. **Con (deck):** converting back to mean and covariance requires a matrix inversion, which can be expensive. So: EIF is better when you *add* a lot and *query* rarely; EKF is better when you must know μ every step. Against GraphSLAM: EIF is the same mathematics; GraphSLAM is the graph-shaped presentation of it, with the constraints made explicit as edges.

6. Best suited when

Full/offline SLAM on a large map with many landmarks that are rarely co-visible — the regime where Ω really is sparse and the inversion is paid once.

7. ML / DNN / RL

Same targets as EKF-SLAM (learned Jacobians, learned Q/R , semantic data association). The information-form-specific one is **learned sparsification**: deciding which weak links to prune, which is what SEIF does with a hand-set rule and what a learned policy can do better.

6.3 SEIF (Sparse Extended Information Form)

1. What it is

EIF made genuinely online by enforcing sparsity rather than hoping for it. As the robot moves, the information matrix fills in with weak links between the robot and every landmark it has ever seen; SEIF adds a *sparsification* step that deactivates a feature by eliminating its link to the robot, bounding the number of non-zero entries per row and so keeping the update cost constant.

2. Canonical formulas

The same Ω/ξ recursion as EIF, plus, after each correction, a sparsification projection that removes selected off-diagonal entries and redistributes their information so the resulting Gaussian remains a valid (conservative) approximation. Formally, the link between robot and a deactivated feature is set to zero and the marginal is preserved on the remaining active set.

3. Algorithm

Algorithm

```
SEIF-SLAM( $\Omega$ ,  $\mathbf{x}_i$ ,  $\mathbf{u}_t$ ,  $\mathbf{z}_t$ ):
1. Initialization
2. Prediction: apply motion model, predict pose, update information matrix
3. Correction (a): apply the measurement model
4. Correction (b): data association
5. Correction (c): SPARSIFICATION - deactivate a feature by eliminating its
link to the robot
6. Correction (d): update pose, map and information; repeat
```

4. Worked example

Exam Tip / Worked Example

Continue the EIF result. After the update,

$$\Omega = \begin{pmatrix} 2.667 & -2.0 & 0 \\ -2.0 & 2.25 & 0 \\ 0 & 0 & 0.25 \end{pmatrix}$$

Suppose the robot now observes L_2 as well, creating a link $\Omega_{r,m_2} = -1.5$ and raising Ω_{rr} to 4.167 and $\Omega_{m_2m_2}$ to 1.75 (an $H^T R^{-1} H$ addition of the same shape). The matrix is now *fully dense in the robot row*:

$$\Omega = \begin{pmatrix} 4.167 & -2.0 & -1.5 \\ -2.0 & 2.25 & 0 \\ -1.5 & 0 & 1.75 \end{pmatrix}$$

With N landmarks the robot row has N non-zeros and each update touches all of them — the cost is no longer constant.

Sparsify: deactivate L_2 (the weaker link, $|-1.5| < |-2.0|$) by zeroing Ω_{r,m_2} and conservatively reducing Ω_{rr} by the information that link carried, $\Omega_{r,m_2}^2 / \Omega_{m_2m_2} = 1.5^2 / 1.75 = 1.286$:

$$\Omega_{sparse} = \begin{pmatrix} \mathbf{2.881} & -2.0 & \mathbf{0} \\ -2.0 & 2.25 & 0 \\ \mathbf{0} & 0 & 1.75 \end{pmatrix}$$

The robot row is back to one active link. L_2 's own information (1.75) is kept, so the landmark is not forgotten — only its *correlation* with the current pose is dropped.

The trade being made: Ω_{rr} fell from 4.167 to 2.881, i.e. the filter now claims *less* certainty about the robot than it strictly has. Sparsification is deliberately conservative — it throws away information rather than inventing it, which keeps the estimate safe (never overconfident) at the cost of some accuracy.

5. Compared with

Comparison

Against EIF: constant-time updates instead of growing ones, at the cost of a conservative (slightly pessimistic) covariance. Against EKF-SLAM: linear rather than quadratic in landmarks, so it reaches map sizes EKF cannot. Against GraphSLAM: SEIF is online and approximate; GraphSLAM is batch and exact, and re-optimises rather than discarding.

6. Best suited when

Large-scale online SLAM on constrained hardware, where the map must keep growing but the per-step budget must not.

7. ML / DNN / RL

The sparsification choice — which links to cut, and when — is a policy decision made today by a fixed rule (keep the k strongest). RL with a reward trading map accuracy against update latency is the natural learned replacement, and is the same shape of problem as the RL beam subsampling in the sensor-model chapter.

6.4 GraphSLAM (pose-graph optimisation)

1. What it is

Stops filtering and starts optimising. Every robot pose and landmark becomes a node; every odometry reading and every observation becomes an edge carrying a constraint. Because odometry drifts, the constraints are mutually inconsistent — the graph does not close. GraphSLAM finds the configuration of all nodes that minimises the total squared constraint violation, so a loop closure corrects the entire trajectory at once rather than only the current pose.

2. Canonical formulas

Each constraint contributes an error term that should be zero:

$$F(y) = \sum_i e_i(y)^2 \quad (\text{with information weights, } \sum_i e_i^\top \Omega_i e_i)$$

Linearise each e_i about the current estimate, $e_i(y + \Delta) \approx e_i(y) + J_i \Delta$, and solve the normal equations

$$\left(\sum_i J_i^\top \Omega_i J_i \right) \Delta = - \sum_i J_i^\top \Omega_i e_i, \quad y \leftarrow y + \Delta$$

iterating to convergence. The matrix on the left is exactly the information matrix Ω of EIF — GraphSLAM and EIF are the same object seen from two sides.

3. Algorithm

Algorithm

```
GraphSLAM(odometry stream, observation stream):
1. Initialization: assume the sensor is perfect, variance = 1; anchor the origin
2. Prediction: simulate the motion model; store each predicted pose as a node
3. Correction (a): apply the measurement model to each observation
4. Correction (b): data association - decide which landmark was seen
5. Correction (c): store the landmark with its correlation link; build the
```

graph

6. Solve: linearise every e_i , assemble the normal equations, solve for the correction,
iterate; loop closures re-optimize every node at once

4. Worked example

Exam Tip / Worked Example

State vector $y = [x_0, x_1, x_2, L_1, L_2]^T$, initial guess $y = [0, 2, 3, 4, 9]^T$ (dead reckoning). Six constraints:

Type	Physical observation	Error equation ($e_i = 0$ if satisfied)
Prior (anchor)	origin is at 0	$e_0 = x_0 - 0$
Motion 1	moved +2 from x_0	$e_1 = x_1 - x_0 - 2$
Motion 2	moved +1 from x_1	$e_2 = x_2 - x_1 - 1$
Measurement 1	spotted L_1 at +2.5 from x_1	$e_3 = L_1 - x_1 - 2.5$
Measurement 2	spotted L_2 at +6.0 from x_2	$e_4 = L_2 - x_2 - 6$
Loop closure	re-spotted L_1 at +1.2 from x_2	$e_5 = L_1 - x_2 - 1.2$

Evaluate the initial guess. $e_0 = 0$, $e_1 = 2 - 0 - 2 = 0$, $e_2 = 3 - 2 - 1 = 0$, $e_3 = 4 - 2 - 2.5 = -0.5$, $e_4 = 9 - 3 - 6 = 0$, $e_5 = 4 - 3 - 1.2 = -0.2$.

$$F(y_0) = 0 + 0 + 0 + 0.25 + 0 + 0.04 = \mathbf{0.29}$$

Dead reckoning satisfies every *motion* constraint perfectly and violates the two *landmark* constraints — the signature of odometry drift.

Solve the least-squares problem (all information weights = 1):

$$y^* = [0, \mathbf{2.0}, \mathbf{3.1}, \mathbf{4.4}, \mathbf{9.1}]^T$$

Residuals: $e = [0, 0, 0.1, -0.1, 0, 0.1]$, so

$$F(y^*) = 0.01 + 0.01 + 0.01 = \mathbf{0.03}$$

— a tenfold reduction. The optimiser accepted a small motion error ($e_2 = 0.1$) in exchange for a much smaller landmark error, spreading the inconsistency across the whole graph instead of dumping it on the last node.

Errata note: the deck prints $y' = [0, 2.11, 3.18, 4.45, 9.18]$, which gives $F = 0.0475$ — higher than 0.03, so it is one damped Gauss–Newton step rather than the converged optimum. Either is defensible in an exam *if you say which you computed*. See Section 10.

Jacobian, for the linearisation step. With y ordered as above, $\partial e_3 / \partial y = [0, -1, 0, 1, 0]$ — e_3 involves only x_1 and L_1 . Every row of J is this sparse, and that sparsity is what makes GraphSLAM scale.

5. Compared with

Comparison

The full comparison table is under EKF-SLAM above. In one line: GraphSLAM trades online recursion for global optimality — it is not incremental, but it corrects drift across the entire trajectory and handles loop closure explicitly, which is precisely what a filter cannot do. Memory is lower (sparse structure) and accuracy higher in long-term mapping.

EC3 Q1(b)(i) — making GraphSLAM beat FastSLAM

FastSLAM is online, particle-based, and forgets: a pose sampled and resampled away is gone. It suits a robot driving forward once through an unknown space. To make GraphSLAM the better answer, modify the scenario so that the robot *revisits* places: give it a search pattern that sweeps each floor and returns past the information kiosk several times, or send it in and back out along a partly different route. Now there are genuine loop closures, and the accumulated drift over a long traverse is exactly the error GraphSLAM removes globally — it re-optimises every past pose at once, whereas FastSLAM can only reweight the particles it still has, and by then the correct hypothesis has usually been resampled out of existence. Add that the rescue team needs an accurate *map* afterwards (to route human teams), not just a live pose, and the batch/offline nature of GraphSLAM stops being a drawback.

6. Best suited when

Large environments, long traverses with revisits, and a requirement for a globally consistent map — especially when the map is a deliverable in its own right and can be optimised offline.

7. ML / DNN / RL**Architecture / Implementation**

Loop-closure detection is the learning-heavy part: CNN place-recognition embeddings (NetVLAD, DBoW2 with CNNs) decide whether two images are the same place, which is what creates the e_5 -style constraint in the first place. **Front-end features:** SuperPoint, D2-Net and R2D2 replace hand-crafted ORB/SIFT. **Robust back-end:** learned robust kernels down-weight edges that are probably false loop closures, so one bad association does not drag the whole graph.

6.5 FastSLAM (Rao–Blackwellised particle filter)**1. What it is**

Exploits a structural fact: *if you knew the robot's whole path, the landmarks would be independent of one another*. So sample the path with particles, and give each particle its own tiny independent EKF per landmark. The joint problem factorises into one high-dimensional sampling problem and many one-dimensional Gaussian problems, which is why it scales where EKF-SLAM does not — and it also means data association is decided *per particle*, so a wrong guess only kills that particle instead of corrupting the whole map.

2. Canonical formulas

Factorisation:

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) = p(x_{1:t} \mid z_{1:t}, u_{1:t}) \prod_i p(m_i \mid x_{1:t}, z_{1:t})$$

Each particle $P^k = (x^k, \mu_1^k, \Sigma_1^k, \mu_2^k, \Sigma_2^k, \dots, w^k)$. Per-particle landmark EKF, with H_m the landmark Jacobian:

$$S^k = \Sigma_i^k + R, \quad K^k = \Sigma_i^k H_m (S^k)^{-1}, \quad \mu_i^k \leftarrow \mu_i^k + K^k E^k, \quad \Sigma_i^k \leftarrow (1 - K^k H_m) \Sigma_i^k$$

Particle weight:

$$w^k = \frac{1}{\sqrt{2\pi S^k}} \exp\left(-\frac{(E^k)^2}{2S^k}\right)$$

3. Algorithm

AlgorithmFastSLAM(particle set, u_t , z_t):

1. Initialization: each particle $P^k = (\text{pose, per-landmark mean and variance, weight})$
2. Prediction: apply the motion model, SAMPLING noise per particle - poses now differ
3. Correction (a): measurement model, per particle
4. Correction (b): data association, INDEPENDENTLY per particle
5. Correction (c/d): per-particle EKF update of the matched landmark; compute weight w^k
6. Resample particles proportional to normalised w^k ; repeat

4. Worked example — the EC3 exam question, recomputed**Exam Tip / Worked Example**

Setup (EC3 Q1a). Two particles, hypotheses *before* motion: P_1 at 6 m, P_2 at 4 m. Control $u = +3$ m. LiDAR returns two range measurements $Z_1 = 1$ m, $Z_2 = 12$ m. Simplified model: landmarks at $L_1 = 10$, $L_2 = 20$; gain $K = 1$; $S^{-1} = 0.5$. Weight $w = \exp(-S^{-1} \sum_j E_j^2)$.

Step 2 — Prediction (motion model).

$$x^{(1)} = 6 + 3 = \mathbf{9}, \quad x^{(2)} = 4 + 3 = \mathbf{7}$$

Step 3a — Measurement model. Predicted range $\hat{z}_j = L_j - x$:

Particle	$\hat{z}_1 = L_1 - x$	$E_1 = Z_1 - \hat{z}_1$	$\hat{z}_2 = L_2 - x$	$E_2 = Z_2 - \hat{z}_2$
P_1 (at 9)	$10 - 9 = 1$	$1 - 1 = \mathbf{0}$	$20 - 9 = 11$	$12 - 11 = +1$
P_2 (at 7)	$10 - 7 = 3$	$1 - 3 = -2$	$20 - 7 = 13$	$12 - 13 = -1$

Step 3d — Per-particle landmark update, $\mu_j \leftarrow \mu_j + KE_j$ with $K = 1$:

Particle	updated L_1	updated L_2
P_1	$10 + 1(0) = \mathbf{10}$	$20 + 1(+1) = \mathbf{21}$
P_2	$10 + 1(-2) = \mathbf{8}$	$20 + 1(-1) = \mathbf{19}$

Each particle now carries its own, different map — this is the defining property.

Step 5 — Weights.

$$w^{(1)} = \exp(-0.5(0^2 + 1^2)) = e^{-0.5} = \mathbf{0.6065}$$

$$w^{(2)} = \exp(-0.5((-2)^2 + (-1)^2)) = e^{-2.5} = \mathbf{0.0821}$$

Normalised:

$$\bar{w}^{(1)} = \frac{0.6065}{0.6065 + 0.0821} = \frac{0.6065}{0.6886} = \mathbf{0.881}, \quad \bar{w}^{(2)} = \mathbf{0.119}$$

Answer to state: P_1 carries 88.1% of the belief with map $\{L_1 = 10, L_2 = 21\}$; P_2 carries 11.9% with $\{L_1 = 8, L_2 = 19\}$. On resampling, P_1 will be duplicated and P_2 will most likely die.

Two traps in that question. (i) The paper says the measurements are “relative to the global frame”, but only the *range* interpretation is numerically consistent — treat Z_j as ranges and say so. (ii) $Q = 1$ and $R = 2$ appear in the figure and are **distractors**: the simplified model hands you K and S^{-1} directly, so neither is needed. Do not spend time deriving a Kalman gain you were given.

5. Compared with

Comparison

Against EKF-SLAM: scales far better (no dense $N \times N$ covariance), tolerates non-linear motion because poses are sampled rather than linearised, and — the big one — resolves data association per particle, so ambiguity is represented instead of being resolved wrongly and permanently. Against GraphSLAM: online and incremental, but no global re-optimisation, so long-term drift is corrected only weakly and only if the correct particle survived. Against plain MCL: FastSLAM *is* MCL with a map attached to every particle.

Limitation / Warning

Particle depletion is the characteristic failure. Resampling repeatedly kills hypotheses; over a long traverse the particle set loses diversity, and once the correct pose history has been resampled away it cannot come back. This is exactly why loop closure is weak in FastSLAM and strong in GraphSLAM — and it is the reason the EC3 answer to Q1(b)(i) is GraphSLAM.

6. Best suited when

Feature-rich environments with ambiguous data association, non-linear motion, large landmark counts, and a mission that moves forward rather than revisiting — and where an online estimate matters more than a globally optimal map.

7. ML / DNN / RL

Architecture / Implementation

Same learned-proposal idea as the particle filter: a network that proposes poses conditioned on z_t dramatically reduces the particle count and therefore the depletion risk. **Visual FastSLAM** is the deck's practical form — CNN feature extraction (SuperPoint, D2-Net) instead of hand-crafted corners, monocular depth (Monodepth, PackNet, DPT), learned visual odometry (DeepVO, VINet), and CNN place recognition (NetVLAD) to catch the loop closures the particle filter would otherwise miss.

6.6 Visual SLAM

1. What it is

SLAM with a camera as the primary sensor instead of LiDAR or sonar. It tracks visual features across frames, estimates relative camera motion from their apparent movement, and triangulates a map — sparse or dense — from the accumulated views. Traditional pipelines use hand-crafted features (ORB, SIFT), epipolar geometry for pose, bundle adjustment for refinement, and bag-of-words image retrieval for loop closure; the modern pipeline replaces each of those stages with a learned module.

2. Canonical formulas

Pinhole projection of a 3-D point X into pixel u : $u = K[R \mid t]X$, with K the camera intrinsics. Bundle adjustment minimises reprojection error over all poses and points:

$$\min_{\{R_i, t_i\}, \{X_j\}} \sum_{i,j} \|u_{ij} - \pi(K[R_i \mid t_i]X_j)\|^2$$

This is GraphSLAM's least-squares objective with reprojection as the error term.

3. Algorithm

Algorithm

VisualSLAM(image stream):

1. extract visual features (corners, blobs) from each frame - ORB/SIFT, or a CNN
2. track how features move across consecutive frames
3. estimate relative camera motion from motion + appearance (epipolar geometry, or DeepVO)
4. triangulate matched features into a sparse or dense 3-D map
5. detect loop closures by image retrieval (bag-of-words, or NetVLAD embeddings)
6. run bundle adjustment over poses and points to refine everything globally

4. Worked example**Exam Tip / Worked Example**

Stereo depth from disparity — the one visual-SLAM calculation that is genuinely hand-sized, and it is on the deck's sensor slide.

$$Z = \frac{f \cdot B}{d}$$

where f is focal length in pixels, B the baseline, d the disparity in pixels.

With $f = 800$ px, $B = 0.1$ m, and a feature displaced by $d = 10$ px between the two views:

$$Z = \frac{800 \times 0.1}{10} = 8 \text{ m}$$

Now move the feature closer so disparity doubles to 20 px: $Z = 80/20 = 4$ m. **Depth is inversely proportional to disparity**, which has two consequences worth stating: distant points have tiny disparities, so their depth is very poorly conditioned (a 1-pixel error at $d = 10$ shifts Z by 0.8 m; at $d = 2$ it shifts Z by 13 m); and a monocular camera, having $B = 0$, cannot recover scale at all — which is why monocular visual SLAM produces a map that is correct up to an unknown scale factor.

5. Compared with

Comparison				
	Visual SLAM	EKF-SLAM	EIF-SLAM	FastSLAM
Sensor	camera (mono, stereo, RGB-D)	LiDAR/sonar + odometry	same	same
Estimation	feature tracking + bundle adjustment	Kalman filter	information filter	particle filter + per-landmark EKF
Map	sparse or dense 3-D from images	landmarks (mean + covariance)	same, information form	landmarks per particle
Loop closure	yes — place recognition + optimisation	weak	possible but expensive	implicit, via particles
Scalability	high (graph-based)	poor	slightly better	better
Drift correction	strong	weak to moderate	same as EKF	moderate
Domain	AR/VR, drones, mobile robots	small-scale navigation	theoretical improvement EKF	robotics, embedded

Limitation / Warning

Drawbacks the deck names explicitly: low light, textureless surfaces, and dynamic scenes. Add scale ambiguity for monocular, and motion blur under fast movement.

6. Best suited when

Cameras are the only affordable or permitted sensor, the scene is well-lit and textured, and appearance-based loop closure is valuable — AR/VR, drones, and consumer robots.

7. ML / DNN / RL

Architecture / Implementation

The deck's seven-row table of where CNNs enter visual SLAM:

Use in SLAM	What the network does	Examples
Feature extraction	learn robust features instead of hand-crafted ones	SuperPoint, D2-Net, R2D2
Depth estimation (mono)	infer depth from a single image	Monodepth, PackNet, DPT
Visual odometry	learn relative motion directly from images	DeepVO, VINet, FlowNet
Loop-closure detection	place recognition via image embeddings	NetVLAD, DBoW2 + CNNs
Semantic mapping	classify objects while mapping	SemanticFusion, MaskFusion
Scene / optical flow	learn 3-D motion between frames	RAFT, PWC-Net
End-to-end SLAM	predict pose and map directly from raw input	Neural SLAM, DVSO, DeepFactors

7 Path Planning I — Foundations, Graph Search, Sampling

Where path planning sits

Perception → Localization → Mapping → **Path Planning** → Control. It is the bridge between “where am I / what is around me” and “how do I actually move”. Two layers, always:

	Global planner	Local planner
Map used	full pre-built map	rolling window around the robot
Goal	best overall route to the goal	safe, feasible motion for the next few seconds
Obstacles	known, static, on the map	sensed, including moving ones
Update rate	low, 0.2–1 Hz, or on map change	high, 5–20 Hz, every control cycle
Algorithms	Dijkstra, A*, D*	DWA, TEB, MPC
Output	a waypoint path	velocity commands (v, ω)

And the planner/controller split: the *planner* answers “what path should I take?” over seconds; the *controller* answers “how do I follow it right now?” over milliseconds, using PID, pure pursuit or MPC.

7.1 Costmaps and path evaluation metrics

1. What it is

A costmap is a grid where each cell stores a *traversal cost*, not just free/occupied. Three layers stack: the static layer from the pre-built map (walls, shelving), the obstacle layer updated live from sensors (people, carts), and the inflation layer that raises cost near obstacles so the planner keeps a margin rather than treating “touching” as the only illegal state. Both planners search the costmap, not the raw map — which is why “cheapest” and “shortest” are not the same path.

2. Canonical formulas

Cost values run 0 (free) to 254 (lethal), with a gradient from inflation in between. Path evaluation

metrics, all of which appear in metaheuristic fitness functions later:

$$\text{Length} = \sum_i \|p_{i+1} - p_i\|, \quad \text{Smoothness} = \sum_i |\theta_{i+1} - \theta_i|, \quad \text{Clearance} = \min_i d(p_i, \mathcal{O})$$

plus computation time, success rate, optimality gap (cost vs. optimal) and nodes expanded. Recommended energy model:

$$E = \sum (e_t \Delta d + e_r |\Delta \theta| + e_p \cdot \text{slope} \cdot \Delta d)$$

with e_t the translational cost per metre, e_r the rotational cost per degree, e_p the terrain/slope penalty.

3. Algorithm

Algorithm

BuildCostmap(static map, live scan):

1. static layer: copy the pre-built map (walls, shelving) - rarely updated
2. obstacle layer: mark cells hit by the current scan - updated every cycle
3. inflation layer: for each obstacle cell, raise the cost of cells within the inflation radius, decaying with distance
4. compose: cell cost = max over layers
5. global planner searches the global (static + inflation) costmap
6. local planner searches the rolling local costmap including live obstacles

4. Worked example

Exam Tip / Worked Example

Three candidate paths through a corridor with a pillar. Weights $w_{len} = 1$, $w_{clear} = 2$ (as an inverse-clearance penalty), $w_{smooth} = 0.5$. Cost = $w_{len}\ell + w_{clear}/\text{clearance} + w_{smooth} \cdot \text{turning}$.

Path	length ℓ	clearance	total turning	Cost
P_A (hugs the pillar)	10.0	0.25 m	30°	$10 + 8 + 15 = \mathbf{33.0}$
P_B (middle)	11.0	1.00 m	20°	$11 + 2 + 10 = \mathbf{23.0}$
P_C (wide detour)	14.0	2.00 m	10°	$14 + 1 + 5 = \mathbf{20.0}$

Check P_A : $1(10.0) + 2(1/0.25) + 0.5(30) = 10 + 8 + 15 = 33.0$. ✓

The shortest path is the worst path. P_A wins on length and loses badly on the other two terms. This is the entire argument for costmaps over binary maps, and it is also why the weights must be chosen deliberately: raise w_{len} to 3 and P_A wins ($30 + 8 + 15 = 53$ vs $33 + 2 + 10 = 45$ vs $42 + 1 + 5 = 48$ — now P_B wins, and at $w_{len} = 5$, P_A finally does). Always state your weights.

5. Compared with

Comparison

Against a binary occupancy map: a costmap lets the planner express preference rather than only legality, which is what produces paths a human would call sensible. Against planning directly on the probability grid: the costmap bakes inflation in once, so the planner does not re-derive clearance at every node expansion.

6. Best suited when

Always, for mobile robots. The interesting question is not whether to use a costmap but how to split static from dynamic: static obstacles are cheap (known in advance, handled by the global planner);

dynamic obstacles are expensive (must be tracked and predicted, handled by the local planner every scan).

7. ML / DNN / RL

Learned costmaps are the natural extension: a network maps raw perception to traversal cost directly, capturing “this looks like loose gravel” or “this doorway is usually crowded at 9am” — semantics no hand-written inflation rule encodes.

7.2 Dijkstra’s algorithm

1. What it is

Uniform-cost search. It expands nodes in strict order of accumulated cost from the start, so the first time it reaches any node it has found the cheapest route there. It uses no information about where the goal is, which makes it complete and optimal but wasteful: it explores outward in all directions equally.

2. Canonical formulas

$g(n)$ = cost of the cheapest known path from start to n . Relaxation:

$$\text{if } g(u) + c(u, v) < g(v) \text{ then } g(v) \leftarrow g(u) + c(u, v), \text{ prev}(v) \leftarrow u$$

The priority queue is keyed on $g(n)$ alone.

3. Algorithm

Algorithm

```
Dijkstra(graph, x_init, x_goal):
1. dist = {x_init: 0}; prev = {}; pq = [(0, x_init)]; visited = empty
2. while pq not empty: (d, u) = pop_min(pq); if u in visited: continue
3.   visited.add(u); if u == x_goal: break
4.   for v in Neighbors(u): nd = d + Cost(u, v)
5.     if v not in dist or nd < dist[v]: dist[v] = nd; prev[v] = u; push
      (nd, v)
6. return ReconstructPath(prev, x_goal)
```

4. Worked example

See the A* example below — the two are run on the same roadmap so the difference in expansions is visible side by side. Dijkstra expands **11** nodes; A* expands **7**; both return the same optimal path of cost 13.264.

5. Compared with

Comparison

Strictly dominated by A* with an admissible heuristic: same optimality guarantee, never more expansions, usually far fewer. Dijkstra is A* with $h \equiv 0$. It remains the right choice when no meaningful heuristic exists (an abstract graph with no geometry), or when you need cheapest paths to *all* nodes rather than to one goal.

6. Best suited when

Small or medium graphs, no usable distance-to-goal estimate, or a single-source all-destinations query.

7. ML / DNN / RL

The learning target is the heuristic — which turns Dijkstra into A*. See below.

7.3 A* search

1. What it is

Dijkstra plus a heuristic estimate of the remaining distance. It orders the queue by $f = g + h$ rather than g alone, so expansion is pulled toward the goal. If h never overestimates the true remaining cost (admissible), A* is still guaranteed to return the optimal path, but reaches it after far fewer expansions.

2. Canonical formulas

$$f(n) = g(n) + h(n)$$

$g(n)$ = cost so far, $h(n)$ = estimated cost to the goal. **Admissible** means $h(n) \leq h^*(n)$ (never overestimates) — this guarantees optimality. **Consistent** means $h(n) \leq c(n, n') + h(n')$ — this additionally guarantees each node is expanded only once. Straight-line Euclidean distance is both, whenever edge costs are actual distances.

3. Algorithm

Algorithm

```
AStar(graph, x_init, x_goal):
1.  g = {x_init: 0}; prev = {}; pq = [(h(x_init), x_init)]; visited = empty
2.  while pq not empty: (f, u) = pop_min(pq); if u in visited: continue
3.      visited.add(u); if u == x_goal: break
4.      for v in Neighbors(u): ng = g[u] + Cost(u, v)
5.          if v not in g or ng < g[v]: g[v] = ng; prev[v] = u; push (ng +
h(v), v)
6.  return ReconstructPath(prev, x_goal)
```

4. Worked example — on the course's own PRM roadmap

Exam Tip / Worked Example

Roadmap. 12 nodes from the deck's PRM* sampling batches, connected by the PRM* radius $r = 8\sqrt{\ln 12/12} = 3.640$ (see PRM* below). Start $S = (1, 1)$, Goal $G = (10.5, 10)$.

$S(1, 1)$	$A(2, 2)$	$B(4, 4)$	$C(5.5, 5.5)$	$D(6.5, 5.5)$	$E(7, 4)$	$K(3.5, 6.5)$
$F(8.5, 8.5)$	$H(9, 9)$	$I(9.5, 9.5)$	$G(10.5, 10)$	$J(11, 10)$		

Heuristic $h(n)$ = Euclidean distance to G : $h(S) = 13.09$, $h(A) = 11.67$, $h(B) = 8.85$, $h(C) = 6.73$, $h(D) = 6.02$, $h(E) = 6.95$, $h(K) = 7.83$, $h(F) = 2.50$, $h(H) = 1.80$, $h(I) = 1.12$, $h(J) = 0.50$, $h(G) = 0$.

Relevant edges: $S-A$ 1.414, $A-B$ 2.828, $B-C$ 2.121, $B-D$ 2.915, $C-D$ 1.000, $D-F$ 3.606, $F-G$ 2.500, $F-H$ 0.707, $H-G$ 1.803.

A* expansion order:

#	node	g	h	$f = g + h$
1	S	0.000	13.086	13.086
2	A	1.414	11.673	13.087
3	B	4.243	8.846	13.089
4	C	6.364	6.727	13.091
5	D	7.158	6.021	13.179
6	F	10.764	2.500	13.264
7	G	13.264	0.000	13.264

Check row 5: $g(D) = g(B) + c(B, D) = 4.243 + 2.915 = 7.158$; $f = 7.158 + 6.021 = 13.179$. ✓ (Reaching D via C costs $6.364 + 1.000 = 7.364$, which is worse, so the $B \rightarrow D$ edge wins.)

Path: $S \rightarrow A \rightarrow B \rightarrow D \rightarrow F \rightarrow G$, cost **13.264**, **7 expansions**.

Dijkstra on the identical graph expands $S, A, B, C, K, D, E, F, H, I, G$ — **11 expansions** — and returns the same path at the same cost. The four extra nodes (K, E, H, I) are ones A^* never touched because their f values exceeded 13.264.

Notice how flat the f column is — 13.086, 13.087, 13.089, 13.091. That is a near-perfect heuristic on a near-straight route: A^* is walking almost directly down the optimal path. A jagged f column means the heuristic is weak and A^* is degenerating toward Dijkstra.

5. Compared with

Comparison

Against Dijkstra: same guarantees, fewer expansions — always prefer it when a heuristic exists. Against weighted A^* ($f = g + \varepsilon h$, $\varepsilon > 1$): weighted A^* is faster still but gives up optimality, returning a path at most ε times optimal. Against sampling planners: A^* is complete and optimal on the graph it is given, but that graph must be built, and in high dimensions building it is impossible — which is exactly the motivation for RRT and PRM. Against D^* : A^* must replan from scratch when the map changes.

6. Best suited when

Small or medium 2-D maps, a known static environment, and a requirement for the provably shortest path. The deck's planner selection table puts Dijkstra/ A^* first for exactly this case.

7. ML / DNN / RL

Architecture / Implementation

The heuristic is the learning target: a network that predicts true cost-to-go from (state, goal, local map) makes A^* expand dramatically fewer nodes. The catch is admissibility — a learned h that overestimates breaks the optimality guarantee, so learned heuristics are usually either clamped to an admissible bound or accepted as bounded-suboptimal. RL sits one level up, learning *which planner to invoke* in which region of the map.

7.4 D^* (dynamic A^*)

1. What it is

A^* for a world that changes while you are driving through it. It plans backwards from the goal and keeps the g values it computed; when an edge cost changes, it repairs only the part of the search tree that the change actually affects rather than throwing the whole plan away. The result is identical to

a full replan but reached with far fewer expansions — which is what makes it viable at the replanning rates a moving robot needs.

2. Canonical formulas

Same $k = f = g + h$ key as A*, but g is cost-to-goal rather than cost-from-start, because the search runs backwards. A node becomes **inconsistent** when its stored g no longer matches the best value obtainable from its successors:

$$g(u) \neq \min_{v \in \text{succ}(u)} (c(u, v) + g(v))$$

Repair means pushing inconsistent nodes back onto the queue and propagating the change outward until consistency is restored — and crucially, propagation stops as soon as the change no longer affects any node's value.

3. Algorithm

Algorithm

```
DStar(graph, x_start, x_goal):
1. compute an initial plan backwards from x_goal (this is A* with g =
   cost-to-goal)
2. the robot begins executing the plan
3. when a sensor reveals a changed edge cost c(u,v):
4.   mark u inconsistent and push it onto the priority queue with key k = g
   + h
5.   pop and repair nodes until every node on the current route is
   consistent again
6. resume execution from the robot's CURRENT position, not from x_start
```

4. Worked example

Exam Tip / Worked Example

Same roadmap. The robot has already driven $S \rightarrow A \rightarrow B \rightarrow D$ and is sitting at D , whose plan was $D \rightarrow F \rightarrow G$ at cost $3.606 + 2.500 = \mathbf{6.106}$.

Event: the sensor reveals that the F - G corridor is blocked by fresh debris. Edge $c(F, G)$ jumps to ∞ .

Repair — expansions from D :

#	node	g from D	h	$k = g + h$
1	D	0.000	6.021	6.021
2	F	3.606	2.500	6.106
3	H	4.313	1.803	6.115
4	G	6.115	0.000	6.115

Check row 3: $g(H) = g(F) + c(F, H) = 3.606 + 0.707 = 4.313$; then $g(G) = 4.313 + c(H, G) = 4.313 + 1.803 = 6.115$. ✓

Repaired plan: $D \rightarrow F \rightarrow H \rightarrow G$, cost **6.115** — **4 expansions**.

Full replan from S with the same blocked edge returns $S \rightarrow A \rightarrow B \rightarrow D \rightarrow F \rightarrow H \rightarrow G$ at cost 13.274, and takes **8 expansions**. D* reached the same repaired route with **half the work**, because it never re-examined the part of the graph behind the robot — S, A, B and the $C/K/E$ side branch were all untouched.

The general principle to state: the saving scales with how localised the change is. A single blocked corridor near the robot is nearly free to repair; a change that invalidates the route near the *start* forces a repair almost as large as a replan.

5. Compared with

Comparison

Against A*: identical result, far cheaper on incremental changes, at the cost of a more complex implementation and memory for the retained search tree. Against a local planner (DWA/TEB): D* still replans a *global* route, so it handles “this corridor is blocked, take another aisle”; DWA handles “a person stepped in front of me, swerve”. They operate at different rates and solve different problems — the deck’s global/local table.

6. Best suited when

Known map, mostly static, with occasional discrete changes discovered during execution — blocked aisles, spills, parked carts. The deck’s selection table lists weighted A* / D* for “known map, but want faster search with good-enough paths”.

7. ML / DNN / RL

Learned heuristics apply as for A*. The D*-specific opportunity is **change prediction**: a model that anticipates which edges are likely to become blocked (a doorway that is busy at shift change) lets the planner pre-emptively prefer robust routes, converting a reactive repair into a proactive choice.

7.5 RRT and RRT*

1. What it is

Sampling-based planning. Instead of building a graph over the whole space, RRT grows a tree from the start by repeatedly drawing a random sample, finding the nearest node in the tree, and steering a bounded step toward it. This explores high-dimensional continuous spaces without ever discretising them. RRT adds *rewiring*: after adding a node it checks whether any nearby node would be cheaper if routed through the new one, and re-parents it if so — which upgrades the guarantee from “finds a path” to “converges to the optimal path”.

2. Canonical formulas

Connection radius, shrinking as the tree grows:

$$r(n) = \min\left(\gamma \left(\frac{\ln n}{n}\right)^{1/d}, \eta\right)$$

n = nodes in the tree, d = dimension, γ = a constant above a percolation threshold, η = the steering step size (the cap).

Choose parent = arg min over x_{near} in the radius of $[\text{Cost}(x_{near}) + c(x_{near}, x_{new})]$.

Rewire: for each x_{near} , if $\text{Cost}(x_{new}) + c(x_{new}, x_{near}) < \text{Cost}(x_{near})$, make x_{new} its parent.

Study Note

Probabilistically complete (RRT and RRT*): the probability of finding a solution, if one exists, tends to 1 as samples $\rightarrow \infty$. It says nothing about any finite run. **Asymptotically optimal** (RRT* only, not RRT): the returned solution converges to the optimum as samples $\rightarrow \infty$. Plain RRT converges to a sub-optimal path almost surely.

3. Algorithm

Algorithm

```

RRT_Star(x_init, n, gamma, eta):
1.  V = [x_init]; E = []
2.  repeat n times:  x_rand = SampleFree(); x_nearest = Nearest(V, x_rand)
3.    x_new = Steer(x_nearest, x_rand, eta)  // move at most eta toward the
    sample
4.    if not ObstacleFree(x_nearest, x_new): continue
5.    X_near = Near(V, x_new, r(n)); choose the parent minimising
    cost-to-come
6.    add x_new; REWIRE each x in X_near whose cost strictly improves through
    x_new

```

4. Worked example — the EC3 exam question, all four iterations**Exam Tip / Worked Example**

Configuration (EC3 Q3a). Start $(0,0)$, step $\eta = 4$, $\gamma = 7$, $d = 2$, **Manhattan distance throughout**. Obstacle cells: collapsed wall $(2,4), (2,5)$; debris pile $(5,2), (6,2)$. A sample is rejected if it coincides with an obstacle cell. Steer: close the x -gap first, then the y -gap, up to a total of η . $r(n) = \min(7(\ln n/n)^{1/2}, 4)$.

Radius table — compute this first, it saves time:

n	raw $7(\ln n/n)^{1/2}$	$r(n)$	note
1	0.000	0.000	$\ln 1 = 0$, so no neighbours at all
2	4.121	4.000	cap binds
3	4.236	4.000	cap binds
4	4.121	4.000	cap binds
5	3.971	3.971	cap finally releases

Iteration 1 — sample $(5,6)$. Tree = $\{(0,0)\}$, $n = 1$.

Nearest = $(0,0)$, Manhattan $d = 5 + 6 = 11 > \eta$.

Steer: x -gap is 5; spend all $\eta = 4$ on $x \Rightarrow x_{new} = (4,0)$, no budget left for y .

$r(1) = 0$, so the candidate set is $\{x_{nearest}\}$ only. **Parent $(0,0)$, cost 4. No rewiring.**

Iteration 2 — sample $(3,2)$. Tree = $\{(0,0), (4,0)\}$, $n = 2$.

Distances: $(0,0) \rightarrow 3 + 2 = 5$; $(4,0) \rightarrow 1 + 2 = 3$. Nearest = $(4,0)$.

$d = 3 \leq \eta$, so the steer reaches the sample: $x_{new} = (3,2)$.

$r(2) = 4$. In-radius: $(0,0)$ is at $5 > 4$ (out); $(4,0)$ at 3 (in). Candidate set = $\{(4,0)\}$.

Parent $(4,0)$, cost $4 + 3 = 7$. No rewiring.

Iteration 3 — sample $(5,2)$. This is a **debris-pile cell**. **Sample rejected**; no node added, n unchanged. *This is deliberate — say so explicitly, it is a marked step.*

Iteration 4 — sample $(2,2)$. Tree = $\{(0,0), (4,0), (3,2)\}$, $n = 3$.

Distances: $(0,0) \rightarrow 4$; $(4,0) \rightarrow 2 + 2 = 4$; $(3,2) \rightarrow 1 + 0 = 1$. Nearest = $(3,2)$.

$d = 1 \leq \eta \Rightarrow x_{new} = (2,2)$. $r(3) = 4$.

Candidate parents (all three are within Manhattan 4):

candidate	cost-to-come	$c(\cdot, x_{new})$	total
$(0,0)$	0	4	4
$(4,0)$	4	4	8
$(3,2)$	7	1	8

Parent = $(0,0)$, cost 4 — *not* the nearest node. That is the whole point of RRT*'s choose-parent step, and it is the mark-bearing observation.

Rewiring check: $(4,0)$ currently costs 4; through x_{new} it would cost $4 + 4 = 8$ — no. $(3,2)$

currently costs 7; through x_{new} it would cost $4 + 1 = \mathbf{5} < 7$ — **REWIRE**: $(3, 2)$'s parent becomes $(2, 2)$, cost $7 \rightarrow 5$.

Final tree:

node	parent	cost-to-come
$(0, 0)$	—	0
$(4, 0)$	$(0, 0)$	4
$(2, 2)$	$(0, 0)$	4
$(3, 2)$	$(2, 2)$	5

The tree never reaches the victim. Part (b) of the exam question asks what that means: probabilistic completeness guarantees only that the probability of finding a path tends to 1 as samples $\rightarrow \infty$; it guarantees *nothing* about four iterations. The practical consequence, given a time-critical rescue, is that stopping here yields no route at all — so the mission needs either an anytime variant that returns the best partial path plus a frontier to continue from, or a fallback planner, rather than a hard iteration cap.

5. Compared with

Comparison

Planner type	Optimality	Completeness	Speed	High dimensions
Graph-based (Dijkstra, A*)	optimal	complete	slow	no
Sampling-based	sub-optimal (RRT) / near-optimal (RRT*)	prob. complete	fast	yes
Optimisation-based	locally optimal	incomplete	varies	yes
Learning-based	sub-optimal	incomplete	very fast	yes
Hybrid	depends on the combination	depends	fast and reactive	yes

RRT vs RRT*: identical exploration, but RRT* pays for choose-parent and rewire on every insertion and gets asymptotic optimality in return. RRT* vs PRM*: RRT* is single-query and grows a tree from the start; PRM* is multi-query and builds a reusable graph.

6. Best suited when

High-dimensional configuration spaces where a grid is impossible — robot arms with many joints — and single-query problems where the map is used once. The deck's table: “RRT* or PRM for high-dimensional space (robot arm, many joints)”.

7. ML / DNN / RL

Architecture / Implementation

Learned sampling is the main lever: instead of sampling uniformly, a generative model trained on past successful paths proposes samples in the narrow passages that matter, cutting the sample count by orders of magnitude. **Learned steering** replaces the geometric steer function with a policy that respects the robot's actual dynamics. **Learned collision checking** (the dominant

cost in RRT*) approximates the obstacle test with a fast network.

7.6 PRM and Batch PRM*

1. What it is

Multi-query sampling. A preprocessing phase scatters samples over the free space and connects each to its neighbours within a radius, producing a reusable roadmap graph; a query phase then answers any start–goal pair with an ordinary graph search over that roadmap. The cost of building the roadmap is paid once and amortised over every subsequent query — which is exactly the opposite trade from RRT*.

2. Canonical formulas

Same shrinking radius as RRT*:

$$r(n) = \gamma \left(\frac{\ln n}{n} \right)^{1/d}$$

with γ a percolation threshold constant (the deck uses ≈ 1.5 or 2 in the generic form, and $\Gamma = 8$ in the worked slides). The $\ln n/n$ shape is what makes PRM* asymptotically optimal: the radius shrinks as samples accumulate, but slowly enough that the graph stays connected.

3. Algorithm

Algorithm

```
BatchPRMStar(n, gamma, x_init, x_goal):
1. V = []; E = [] // LEARNING (preprocessing) PHASE
2. for i in 1..n: x_rand = SampleFree(); r = gamma*(log(i)/i)^(1/d)
3. U = Near(V, x_rand, r); V.append(x_rand); sort U by distance to x_rand
4. for u in U: if CollisionFree(x_rand, u): add edge (x_rand, u)
5. QUERY PHASE: connect x_init and x_goal to the roadmap, then run
   A*/Dijkstra on it
6. return the path; reuse the same roadmap for every future query
```

4. Worked example

Exam Tip / Worked Example

$\Gamma = 8$, $d = 2$, so $r(n) = 8\sqrt{\ln n/n}$. Watch the radius shrink as samples arrive:

n	$r(n)$	
2	$8\sqrt{\ln 2/2} = 8\sqrt{0.3466} = \mathbf{4.710}$	first batch, generous radius
6	$8\sqrt{\ln 6/6} = 8\sqrt{0.2986} = \mathbf{4.372}$	after 5 samples
11	$8\sqrt{\ln 11/11} = 8\sqrt{0.2180} = \mathbf{3.735}$	after the second batch
12	$8\sqrt{\ln 12/12} = 8\sqrt{0.2071} = \mathbf{3.640}$	the roadmap used in the A* example

Check $n = 11$: $\ln 11 = 2.3979$; $2.3979/11 = 0.21799$; $\sqrt{0.21799} = 0.46689$; $8(0.46689) = 3.7352$.
✓

Roadmap construction at $n = 12$, $r = 3.640$. With the 12 nodes listed in the A* example, the pairs within 3.640 become edges: $S-A$ (1.414), $A-B$ (2.828), $B-C$ (2.121), $B-D$ (2.915), $B-E$ (3.000), $B-K$ (2.550), $C-D$ (1.000), $C-E$ (2.121), $C-K$ (2.236), $D-E$ (1.581), $D-K$ (3.162), $D-F$ (3.606), $F-H$ (0.707), $F-I$ (1.414), $F-G$ (2.500), $F-J$ (2.915), $H-I$ (0.707), $H-G$ (1.803), $H-J$ (2.236), $I-G$ (1.118), $I-J$ (1.581), $G-J$ (0.500). **22 edges.**

Rejected as too long: $A-C$ (4.950), $E-F$ (4.743), and every S -to-anything-but- A pair. Note that $D-F$ at 3.606 is the *only* bridge between the left cluster and the right one, and it only just makes the cut — had Γ been 7.5 the radius would be 3.41 and the roadmap would be disconnected. **That fragility is the practical lesson about γ :** too small and the roadmap fragments, too large and the collision-checking cost explodes.

Query phase then runs A^* over these 22 edges, which is the worked example above: 7 expansions, path $S \rightarrow A \rightarrow B \rightarrow D \rightarrow F \rightarrow G$, cost 13.264.

5. Compared with

Comparison

Against RRT*: PRM* wins when the map is static and many queries are asked (build once, query forever); RRT* wins for one-shot planning in a map that changes. Against A^* on a grid: PRM scales to high dimensions where the grid cannot be enumerated, but gives up resolution completeness — a narrow passage may simply never be sampled. Batch vs incremental PRM*: batch draws n samples up front; incremental keeps sampling until a resource limit, recomputing r as n grows.

6. Best suited when

Large, mostly-open continuous spaces with many repeat queries on a fixed map — the deck's example being a warehouse floor plan queried hundreds of times a day. Also humanoid and manipulator planning where the configuration space is high-dimensional but static.

7. ML / DNN / RL

Architecture / Implementation

Learned sampling distributions concentrate roadmap nodes in narrow passages and near likely goal regions, which is where uniform sampling wastes almost all its effort. **Learned collision checking** speeds up the dominant per-edge cost. **Critical-region prediction** — a network that identifies doorways and bottlenecks from the map image — is the most effective published variant, because roadmap quality is decided almost entirely by whether the bottlenecks got sampled.

7.7 Planner selection — the table to memorise

Study Note

Situation	Recommended planner
Small/medium 2-D map, need the provably shortest path	Dijkstra or A*
Known map, want faster search with good-enough paths	Weighted A* / D*
High-dimensional space (robot arm, many joints)	RRT* or PRM
Large, mostly open continuous space, many repeat queries	PRM
Real-time reaction to moving obstacles	DWA, MPPI, or TEB
Car-like robot needing smooth drivable curves	Hybrid A* + MPC
Respect dynamics/limits precisely while tracking	Model Predictive Control
And by platform — same three-layer architecture everywhere, only the algorithms change: warehouse AMR (PRM/A* + DWA/MPPI); autonomous car (Hybrid A* + MPC); robot arm (RRT* + trajectory optimisation); humanoid (PRM/RRT + MPC).	

LLM task planning — where it sits, and where it fails

There are two layers called “planning”. **Task planning** decides what to do and in what order; **motion planning** decides how to physically get there. An LLM turns a natural-language goal (“bring me the red mug from the kitchen”) into an ordered list of sub-goals; each sub-goal is then handed to A*/DWA as before. The LLM **cannot invent a valid path** — its output must be grounded to real map locations, objects and robot skills, and a verification/safety layer must sit between it and the robot because an LLM can propose an infeasible or unsafe step. Full stack: LLM task planner → grounding and safety check → global planner → local planner → controller.

8 Path Planning II — Metaheuristic / Evolutionary Planners

The shared setup — learn this once, it serves all four

All four planners solve the same problem: choose k intermediate way-points between a fixed start S and goal G so as to minimise a cost. The standard cost used across the deck is

$$\text{Cost}(P) = \sum_i \|p_{i+1} - p_i\| + 100 \times (\text{number of colliding segments})$$

$$\text{Fitness}(P) = \frac{1}{\text{Cost}(P)} \quad \text{or} \quad \frac{1}{1 + \text{Cost}(P)}$$

The richer production form is $\text{Cost} = w_1 \text{Length} + w_2 \text{Collisions} + w_3 \text{Smoothness} + w_4 \text{Clearance}^{-1} + w_5 \text{Energy}$, with the rules: normalise each term to a comparable range before weighting; set the collision weight at least an order of magnitude above the largest feasible path length; use $1/(1 + \text{Cost})$ to avoid dividing by zero.

What	differs	is	only	where	the	search	knowledge	lives:
Firefly	in the population	—	each solution is pulled toward brighter neighbours					
GA	in the population	—	solutions recombine and mutate					
PSO	in the population	—	each particle remembers its own best and the swarm’s best					
ACO	in the environment	—	pheromone trails on edges, decaying over time (stigmergy)					

Read this before using any number from the deck's metaheuristic slides

The way-point path lengths printed across the Firefly / GA / ACO / PSO slides do not match the way-points they are computed from. For instance path $P_1 = (1, 1), (3, 2), (6, 4), (9, 8), (10.5, 10)$ is printed as $L = 12.65$ from segments $2.24 + 3.61 + 5 + 1.80$, but the last segment $(9, 8) \rightarrow (10.5, 10)$ is $\sqrt{1.5^2 + 2^2} = 2.50$, not 1.80, giving $L = 13.34$. The same class of error affects P_2 and P_3 . Full audit in Section 10. The worked examples below therefore use either **the EC3 exam's own verified numbers** (ACO) or **independently recomputed examples** (Firefly, GA, PSO). Every figure printed here was recalculated.

8.1 Firefly algorithm

1. What it is

A swarm method where each candidate path is a firefly whose brightness is its solution quality (brighter = lower cost). Every firefly is attracted toward every brighter firefly, and the strength of that attraction falls off exponentially with distance — so a firefly follows its local best rather than jumping straight to the global best. A random jitter term keeps the swarm from collapsing. The absorption coefficient γ is the single hyper-parameter that decides whether it explores or exploits.

2. Canonical formulas

Attractiveness, decaying with distance r :

$$\beta(r) = \beta_0 e^{-\gamma r^2}$$

Movement of firefly i toward brighter firefly j :

$$x_i^{new} = x_i + \beta(r_{ij})(x_j - x_i) + \alpha(\text{rand} - 0.5)$$

β_0 = attractiveness at zero distance; γ = light absorption; α = randomisation scale; $r_{ij} = \|x_i - x_j\|$.
Brightness $\propto 1/\text{Cost}$.

3. Algorithm

Algorithm

```
FireflyAlgorithm(population, gamma, alpha, MaxGeneration):
1. initialise the firefly population and their light intensity from the
   objective function
2. for each generation, for each pair (i, j):
3.     compute distance r; if firefly j is brighter than i:
4.         compute beta = beta0*exp(-gamma*r^2); move i toward j; add
         alpha*(rand-0.5)
5. evaluate the new solutions and update light intensity
6. rank the fireflies, keep the best, increment generation, repeat
```

4. Worked example

Exam Tip / Worked Example

Three candidate paths from $S(1, 1)$ to $G(10.5, 10)$ via three way-points each. Parameters $\beta_0 = 1$, $\gamma = 0.1$, $\alpha = 0.2$, $\text{rand} = 0.3$, $r = \text{Euclidean}$.

Step 1 — brightness. Firefly 2 has way-point (2, 5); Firefly 1 has (3, 2) and is brighter (lower cost). So firefly 2 moves toward firefly 1.

Step 2 — distance between the two way-points.

$$r = \sqrt{(3-2)^2 + (2-5)^2} = \sqrt{1+9} = \sqrt{10} = 3.1623, \quad r^2 = 10$$

Step 3 — attractiveness.

$$\beta = \beta_0 e^{-\gamma r^2} = 1 \cdot e^{-0.1(10)} = e^{-1} = \mathbf{0.3679}$$

Step 4 — jitter. $\alpha(\text{rand} - 0.5) = 0.2(0.3 - 0.5) = \mathbf{-0.04}$.**Step 5 — move.** $\Delta = x_j - x_i = (3 - 2, 2 - 5) = (1, -3)$.

$$x^{\text{new}} = (2, 5) + 0.3679(1, -3) + (-0.04, -0.04) = (2 + 0.368 - 0.04, 5 - 1.104 - 0.04)$$

$$x^{\text{new}} = (\mathbf{2.328}, \mathbf{3.856})$$

The way-point moved about 37% of the way toward the brighter firefly — not all the way, because $\gamma r^2 = 1$ absorbed most of the light.

Now vary γ — this is the exam-worthy part. With the same $r^2 = 10$:

γ	$\beta = e^{-10\gamma}$	behaviour
0.05	0.607	light travels far $\Rightarrow$ wide exploration, but wavy, longer paths
0.30	0.050	balanced — the deck's recommended value
1.00	0.000045	light absorbed instantly $\Rightarrow$ short-sighted, jagged, poor clearance

The deck reports that a balanced $\gamma \approx 0.30$ beats a poorly-tuned one by -21% path length, $3.5\times$ smoother, and $2\times$ obstacle clearance.

Errata note: the deck's slide states $\beta \approx 0.9$ for this move, which requires $r^2 \approx 1.05$, not 10; its $x^{\text{new}} = (2.86, 2.26)$ follows from that. The computation above is the one consistent with the stated γ , β_0 and way-points. In an exam, write the β formula and substitute explicitly — the method earns the marks.

5. Compared with

Comparison

Against GA: firefly moves a *whole path* toward a brighter path; GA recombines way-points from two parent paths. Firefly therefore needs a continuous, meaningful notion of “between two solutions”, which GA does not. Against PSO: both are continuous swarm methods, but PSO uses velocity with memory (personal best + global best) while firefly is memoryless and uses pairwise attraction — so firefly is more local and less prone to premature convergence on one leader. Against ACO: firefly has no persistent memory in the environment; if the map changes, the whole population must re-converge.

6. Best suited when

Smooth, continuous cost surfaces where attraction gives fast early convergence — the deck's own summary. Good for refining an already-reasonable path in a continuous space; poor when the problem is naturally discrete or heavily constrained.

7. ML / DNN / RL

Same lever as ACO below: γ and α are fixed after manual tuning, but the best setting changes during the search (explore early, exploit late). An RL agent with state (iteration number, improvement rate, population diversity), actions (nudge γ , α), and reward (faster discovery of a better solution) makes the algorithm self-tuning.

8.2 Genetic Algorithm for path planning

1. What it is

Evolves a population of candidate paths. Each path is a chromosome; selection favours fitter paths, crossover recombines way-points from two parents, mutation injects way-points that were never in the initial population, and elitism guarantees the best-so-far is never lost. Its distinctive strength is that the chromosome can be *anything* — continuous coordinates, discrete moves, mixed types — which is why it handles constraints that a continuous swarm method cannot express.

2. Canonical formulas

$$\text{Fitness}(P) = \frac{1}{\text{Cost}(P)}, \quad \text{selection probability}(P_i) = \frac{\text{Fitness}(P_i)}{\sum_j \text{Fitness}(P_j)}$$

Gaussian mutation of one gene: $x' = x + \mathcal{N}(0, \sigma)$. Standard hyper-parameters (deck): population 50–200; crossover rate 0.6–0.9; mutation rate 0.01–0.1 per gene; elitism count 1–2.

3. Algorithm

Algorithm

```
GeneticAlgorithm(population size, crossover rate, mutation rate, elite count):
1. initialise the population of chromosomes (each = one candidate path)
2. evaluate fitness = 1 / Cost for every chromosome
3. select parents by roulette wheel, with probability proportional to fitness
4. crossover: single-point swap of gene segments between two parents -> two children
5. mutate: perturb a random gene (Gaussian for coordinates, re-draw for discrete)
6. replace the worst individuals with the children, COPY the elite unchanged; repeat
```

4. Worked example — discrete move-sequence GA

Exam Tip / Worked Example

Problem. 10×10 grid, $S = (0, 0)$, $G = (9, 9)$, obstacles are solid blocks.

Chromosome design. A path is a sequence of moves. **Gene alphabet:** 0 = Up, 1 = Down, 2 = Left, 3 = Right. **Length 25** — the direct route needs 18 moves (9 down, 9 right), and the extra 7 give room for detours around obstacles.

Fitness function.

$$\text{Cost} = W_{\text{dist}} \cdot \text{DistanceToGoal} + W_{\text{len}} \cdot \text{PathLength} + W_{\text{col}} \cdot \text{Collisions}$$

with $W_{\text{dist}} = 5$ (push toward the goal), $W_{\text{len}} = 1$ (prefer short), $W_{\text{col}} = 1000$ (massive penalty for invalid moves). $\text{Fitness} = 1/\text{Cost}$.

Evaluate chromosome $C_2 = [3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 2, 0, 3, 1, 1, 1, 3, 3, 3, 3, 3, 0, 0, 0, 0]$:

Trace: $S(0, 0) \rightarrow R(0, 1) \rightarrow R(0, 2) \rightarrow R(0, 3) \rightarrow D(1, 3) = \text{COLLISION}$ (obstacle).

PathLength = 4 valid moves; Collisions = 1; final point = (1, 3).

$$\text{DistanceToGoal} = \sqrt{(1-9)^2 + (3-9)^2} = \sqrt{64 + 36} = \sqrt{100} = 10.0$$

$$\text{Cost} = 5(10.0) + 1(4) + 1000(1) = 50 + 4 + 1000 = 1054$$

$$\text{Fitness} = 1/1054 = 0.000949$$

Population fitness and roulette shares:

Chrom.	final pt	len	coll.	dist	Cost	Fitness (share)
C_1	(9,9)	18	0	0.00	18	0.055556 (95.11%)
C_2	(1,3)	4	1	10.00	1054.0	0.000949 (1.62%)
C_3	(1,4)	5	1	9.43	1052.2	0.000950 (1.63%)
C_4	(3,3)	6	1	8.49	1048.4	0.000954 (1.63%)

The $1000\times$ collision penalty makes the one valid path $58\times$ fitter than the rest, so it takes 95% of the roulette wheel. **That is the penalty doing its job** — and it is also the risk: with the wheel this skewed, diversity collapses in one generation.

Crossover (single point at index 10), parents C_1 and C_2 :

Child 1 = $C_1[0:10] + C_2[10:]$; Child 2 = $C_2[0:10] + C_1[10:]$. Child 2 inherits C_1 's good tail and often reaches the goal.

Mutation: flip Child 1's gene at index 12 from 3 (Right) to 1 (Down). Mutation is the *only* operator that can create a move never present in the initial population — essential for escaping obstacle-shaped local minima.

Replacement (elitism): the two mutated children replace C_3 and C_4 , the two worst. C_1 survives untouched, so the best-so-far can never be lost. New population = [C_1 , C_2 , Child 1^{mut}, Child 2^{mut}].

EC3 Q2(b) — design a chromosome, gene, and a fitness that is NOT path cost

The question explicitly forbids reusing the path-cost fitness from part (a). A defensible answer for the rescue robot, in five sentences:

- **Chromosome:** an ordered sequence of k intermediate way-points (or corridor IDs) between the command post and the victim, variable length so detours around collapsed sections are representable.
- **Gene:** one way-point — for a graph formulation, the identifier of the next corridor junction; for a continuous formulation, its (x, y) coordinates.
- **Fitness (not path cost):** *expected survival-weighted information gain per unit risk* —

$$\text{Fitness} = \frac{w_1 \cdot \text{VictimReachProbability} + w_2 \cdot \text{UnexploredAreaCovered}}{w_3 \cdot \text{StructuralRisk} + w_4 \cdot \text{SmokeExposureTime} + \varepsilon}.$$
- **Justification:** in a collapsed building the shortest route is frequently the one through the debris pile, so optimising length alone kills the robot; structural risk and smoke exposure are the operative constraints, and unexplored-area coverage matters because the victim's exact position is uncertain. The denominator terms are the ones the pre-disaster map cannot predict, which is precisely why an evolutionary search over whole routes — rather than a greedy per-edge cost — is appropriate.

5. Compared with**Comparison**

GA's edge over Firefly and PSO: it works with discrete or mixed chromosome designs, handles many constraints, and parallelises fitness evaluation trivially. Its weakness against ACO: no persistent memory — if the debris field shifts, GA restarts from a population encoded for the old map, whereas ACO's pheromone decays and re-routes automatically. Against A*/RRT*: no optimality or completeness guarantee at all, and the deck's honest note is that “pure evolutionary is rarely optimal for robotics” — prefer hybrids.

6. Best suited when

The cost function is non-differentiable, multi-objective, or full of hard constraints; the representation is discrete or mixed; and you can afford many fitness evaluations (offline, or with a small population on an embedded target).

7. ML / DNN / RL

Architecture / Implementation

Hybrid GA + local search (memetic): GA for global exploration, then gradient descent or B-spline smoothing to refine the best individuals — faster convergence to high-quality paths.

Neuroevolution: evolve the architecture or weights of a planning network rather than the path itself.

GA as a data generator: evolve high-quality paths offline, then train a supervised network on them, so the deployed robot gets GA-quality paths at network speed.

Dynamic environments: preserve elites from the previous iteration and inject new random chromosomes to explore changed regions; warm-start from the previous solution; implement *anytime* behaviour so the best solution so far is returned when time expires.

8.3 Ant Colony Optimisation

1. What it is

The one method in this group whose memory lives in the environment rather than the population. Ants build paths probabilistically, biased by pheromone (learned desirability) and a heuristic (static attractiveness, usually inverse distance to the goal). After each iteration every trail evaporates, then successful ants deposit pheromone in inverse proportion to their tour cost. The consequence — and the reason ACO is the right answer for shifting environments — is that a stale trail is *forgotten automatically* without any restart or re-encoding.

2. Canonical formulas

Transition probability for an ant at node i choosing node j :

$$p_{ij} = \frac{\tau_{ij}^{\alpha} \eta_{ij}^{\beta}}{\sum_{l \in \text{allowed}} \tau_{il}^{\alpha} \eta_{il}^{\beta}}$$

Evaporation, applied to every edge: $\tau_{ij} \leftarrow (1 - \rho) \tau_{ij}$.

Deposition, by each successful ant k : $\tau_{ij} \leftarrow \tau_{ij} + Q/f_k$, where f_k is ant k 's tour cost.

Parameters: α = pheromone influence (high α exploits, risks premature convergence); β = heuristic influence (high β is greedy toward the goal and blind to obstacles behind it); ρ = evaporation rate; Q = deposit constant; $\eta_{ij} = 1/\text{cost}$ or $1/d(j, G)$.

3. Algorithm

Algorithm

```
AntColonyOptimisation(graph, m ants, alpha, beta, rho, Q, tau0):
1. represent the solution space as a construction graph; set tau = tau0 on every edge
2. each of the m ants walks S -> G, choosing the next node with probability p_ij
3. evaluate each completed tour's cost f_k; invalid tours deposit nothing
4. EVAPORATE: tau_ij = (1 - rho) * tau_ij on EVERY edge
```

5. DEPOSIT: for each successful ant k , $\tau_{ij} += Q / f_k$ on the edges it used
6. repeat until convergence or the stopping criterion is met

4. Worked example — the EC3 exam question, one full iteration

Exam Tip / Worked Example

Setup (EC3 Q2a). Five alternative rescue routes, each through exactly one intermediate way-point. Initial pheromone $\tau = 1$ on every route; evaporation $\rho = 0.2$. Q is not printed — **state your assumption**: take $Q = 1$. Costs: $A = 10$, $B = 8$, $C = 5$, $D = 12$, $E = 15$.

Step 1 — Evaporate every edge.

$$\tau \leftarrow (1 - 0.2)(1) = \mathbf{0.8} \quad \text{on all five routes}$$

Step 2 — Deposit $\Delta\tau = Q/f_k$ on the route each ant used.

Route	cost f_k	$\Delta\tau = 1/f_k$	$\tau_{new} = 0.8 + \Delta\tau$
A	10	0.1000	0.9000
B	8	0.1250	0.9250
C	5	0.2000	1.0000
D	12	0.0833	0.8833
E	15	0.0667	0.8667
Sum			4.5750

Step 3 — Next ant's choice. No heuristic η is given, so the policy is pheromone-only ($p = \tau / \sum \tau$):

	A	B	C	D	E
p	19.67%	20.22%	21.86%	19.31%	18.94%

Answer: the next ant most likely selects waypoint **C**, because it is the cheapest route ($f = 5$) and therefore received the largest deposit (0.2).

The sentence that earns the justification mark: the lead is *tiny* — 21.9% against 18.9% for the worst route. After one iteration with $Q = 1$ the pheromone is still dominated by the 0.8 evaporation residue, so the colony has barely begun to discriminate. Convergence requires many iterations, or a larger Q : with $Q = 100$ the same single iteration gives $P(C) = \mathbf{33.8\%}$, a decisive lead. Q sets how fast the colony commits.

5. Compared with

Comparison

The distinguishing property is **stigmergy**: knowledge lives in the environment, not in the individuals. Consequences — (i) built-in adaptivity: if an obstacle moves, evaporation erases the stale trail while ants naturally re-route, with no restart and no re-encoding; (ii) it needs a *graph*, so a continuous way-point space must be discretised first, and **ACO cannot beat the resolution of its graph**; (iii) it shares knowledge across independent searchers for free, which GA/PSO/Firefly do not. Deck's own rule: *ACO for grid or roadmap graphs and repeated queries on a static map; GA and firefly for continuous refinement.*

EC3 Q3(c) — RRT* vs ACO for a multi-robot, shifting-debris mission

ACO is better suited, on both axes the question names. *Scaling to simultaneous searches:* ACO is natively a population of independent agents sharing one pheromone structure, so n robots searching n debris zones is the algorithm's default mode and their discoveries reinforce each other's trails; RRT* grows one tree toward one goal, so n robots means n independent trees with no mechanism to share what they learn. *Adapting as the debris field shifts:* evaporation is a continuous forgetting mechanism — a route that becomes blocked simply stops being reinforced and decays out within a few iterations, while ants explore around the new geometry; RRT*'s tree encodes a fixed obstacle set, and a shifted obstacle invalidates every branch through it, forcing a rebuild. The honest caveat worth one sentence: ACO needs the corridors discretised into a graph first, and its solution can be no finer than that graph.

6. Best suited when

The problem is naturally a graph (a corridor network, a roadmap, a grid), the same map is queried repeatedly, multiple agents search at once, and the environment shifts slowly enough that evaporation can track it.

7. ML / DNN / RL**Architecture / Implementation**

The deck's headline ACO+RL integration — learn to adapt (α, β, ρ) . The best parameter setting changes *during* the search: early on, encourage exploration (higher β); later, encourage exploitation of strong trails (higher α). Fixed manually-tuned values cannot do both.

- **State:** current iteration number, improvement rate of the best path, diversity of the ants' paths.
- **Action:** slightly increase or decrease α, β, ρ .
- **Reward:** positive if the action leads to faster discovery of a better solution over the next few iterations.
- **Result:** a self-tuning ACO whose exploration/exploitation schedule is learned per problem instance.

ACO with neural heuristics: train a network to predict η from obstacle proximity and goal direction, replacing the simple $1/\text{distance}$ — the deck reports dramatic improvement in solution quality and convergence speed.

8.4 Particle Swarm Optimisation**1. What it is**

Each candidate path is a particle with a position (the way-point vector) and a velocity. Every iteration the velocity is updated by three pulls: inertia (keep going), the cognitive term (back toward this particle's own best-ever position), and the social term (toward the swarm's best-ever position). Positions are then nudged as a continuous vector, so the swarm converges on a smooth region without any graph or encoding.

2. Canonical formulas

$$V_i^{t+1} = wV_i^t + c_1r_1(pBest_i - X_i^t) + c_2r_2(gBest - X_i^t)$$

$$X_i^{t+1} = X_i^t + V_i^{t+1}$$

w = inertia weight; c_1 = cognitive coefficient; c_2 = social coefficient; $r_1, r_2 \sim U(0,1)$. Velocities are clamped to v_{max} per dimension to stop the swarm exploding. Typical values (deck): inertia 0.4–0.9, $c_1 = c_2 = 2.0$.

3. Algorithm

Algorithm

```
PSO(swarm size, w, c1, c2, v_max):
1.  initialise particle positions  $X_i$  and velocities  $V_i$  at random
2.  evaluate fitness (= 1/Cost) for every particle
3.  update pBest_i (this particle's best-ever) and gBest (the swarm's
    best-ever)
4.   $V_i = w*V_i + c1*r1*(pBest_i - X_i) + c2*r2*(gBest - X_i)$ ; clamp to  $v_{max}$ 
5.   $X_i = X_i + V_i$ 
6.  repeat from step 2 until convergence
```

4. Worked example

Exam Tip / Worked Example

Setup. 10×10 grid. Start $S = (1,1)$, Goal $G = (9,9)$. One circular obstacle centred at $C = (5,5)$ with radius $r = 2$. Each particle is a single way-point. Path = $S \rightarrow$ way-point $\rightarrow G$. Cost = path length + 100 if the way-point falls inside the obstacle circle. Parameters $w = 0.5$, $c_1 = 0.8$, $c_2 = 0.9$.

Iteration 1 — initialise and evaluate.

Particle A: position $(4,5)$, velocity $(1,-1)$.

Particle B: position $(7,2)$, velocity $(-1,1)$.

Cost of A: $\|(4,5) - (1,1)\| = \sqrt{9+16} = 5.000$; $\|(9,9) - (4,5)\| = \sqrt{25+16} = 6.403$; length = 11.403. Is A inside the circle? $\|(4,5) - (5,5)\| = 1 < 2 \Rightarrow$ **yes**, add 100.

$$\text{Cost}(A) = \mathbf{111.403}$$

Cost of B: $\|(7,2) - (1,1)\| = \sqrt{36+1} = 6.083$; $\|(9,9) - (7,2)\| = \sqrt{4+49} = 7.280$; length = 13.363. $\|(7,2) - (5,5)\| = \sqrt{4+9} = 3.606 > 2 \Rightarrow$ no penalty.

$$\text{Cost}(B) = \mathbf{13.363}$$

Bests. $pBest_A = (4,5)$, $pBest_B = (7,2)$; since $13.363 \ll 111.403$, **gBest = (7,2)**.

Iteration 2 — update with $r_1 = 0.5$, $r_2 = 0.6$.

For A, the cognitive term is zero ($pBest_A = X_A$), so only inertia and the social pull act:

$$\begin{aligned} V_A^{new} &= 0.5(1, -1) + 0.8(0.5)(0, 0) + 0.9(0.6)[(7, 2) - (4, 5)] \\ &= (0.5, -0.5) + 0.54(3, -3) = (0.5 + 1.62, -0.5 - 1.62) = \mathbf{(2.12, -2.12)} \\ X_A^{new} &= (4, 5) + (2.12, -2.12) = \mathbf{(6.12, 2.88)} \end{aligned}$$

New cost of A: length = $\|(6.12, 2.88) - (1, 1)\| + \|(9, 9) - (6.12, 2.88)\| = 5.452 + 6.766 = 12.218$; $\|(6.12, 2.88) - (5, 5)\| = 2.40 > 2$, no penalty. Cost(A) = **12.218 — a new gBest**.

For B, both $pBest$ and $gBest$ equal its own position, so only inertia acts:

$$V_B^{new} = 0.5(-1, 1) = (-0.5, 0.5), \quad X_B^{new} = \mathbf{(6.5, 2.5)}$$

Read the result. Particle A made a large jump from $(4,5)$ to $(6.12, 2.88)$, *escaping the obstacle* by being pulled toward the much better global best — and in doing so it overshot past $gBest$ and found something better still ($12.218 < 13.363$). Particle B, already the best, merely drifted on its own inertia. That asymmetry — bad particles move far, good particles move little — is the mechanism of PSO in one example.

5. Compared with

Comparison

Against GA: no crossover or mutation, so no discrete/mixed representations, but far fewer moving parts and typically faster convergence on smooth continuous problems. Against Firefly: PSO has memory ($pBest$, $gBest$) and firefly does not; PSO is pulled by a single global leader, which converges faster but is more prone to premature convergence in a multimodal landscape. Against ACO: PSO needs no graph and works directly in continuous way-point space, but has no mechanism for forgetting a stale solution when the map changes.

6. Best suited when

Continuous way-point optimisation on a smooth cost surface, a moderate number of dimensions, and a need for fast convergence with very few hyper-parameters.

7. ML / DNN / RL

Same self-tuning idea as ACO and Firefly: w , c_1 , c_2 are usually fixed but should schedule (high inertia early for exploration, low inertia late for refinement), and an RL controller can learn that schedule. Beyond that, PSO is often used to *train* models rather than the reverse — e.g. optimising the weights of a small planning network where the loss is non-differentiable.

8.5 Metaheuristic planners — the comparison to have ready

Study Note

	Where knowledge lives	Move operator	Distinctive strength
Firefly	population	pairwise attraction $\beta_0 e^{-\gamma r^2}$	smooth continuous surfaces; fast early convergence
GA	population	crossover + mutation	discrete/mixed encodings; many constraints; parallel fitness
ACO	environment (pheromone)	probabilistic construction on a graph	adapts to a changing map for free; multi-agent by nature
PSO	population (with memory)	velocity toward $pBest$ and $gBest$	fewest parameters; fast on continuous smooth costs

Practical realities from the deck. Complexity is $O(G \times P \times N)$ — generations $\times$ population $\times$ path length. For 50 generations, population 100, 20 way-points, that is 100,000 evaluations, taking 1–5 s on an embedded system. For navigation at 0.5 m/s the planner needs at least 1 Hz and ideally 5–10 Hz, so pure evolutionary planning *does not meet real-time requirements*. The fixes are hierarchical planning (fast global + evolutionary local), anytime algorithms, hardware acceleration, small populations (20–30), early stopping, and GPU parallelisation.

Case study (warehouse AMR, 50×30 m, 200+ shelf units). Hybrid chosen: **A*** for the global roadmap (fast, complete) + **GA** for trajectory smoothing with kinodynamic constraints — because A* alone produces jagged paths and pure GA is too slow for full planning. Grid resolution 0.2 m for A*; GA over 15 control way-points; population 30, 20 generations; fitness = path length + smoothness penalty + collision cost + kinematic violation penalty. Result: path quality +35% over pure A*, success rate 98.7%, computation 150 ms (down from 450 ms after pre-computed distance transforms gave a 10 $\times$ speedup, plus vectorised and parallel

fitness evaluation). The named early failure was **population diversity collapse**, fixed with fitness sharing, niching and crowding distance — worth 40% improvement in final path quality.

9 Learning-Based Path Planning

9.1 Reinforcement learning path planner (Q-learning)

1. What it is

Learns a navigation policy by trial and error instead of searching a map. The robot is an agent in an MDP: it observes a state, takes an action, receives a reward, and updates a value estimate for that state-action pair. Over many episodes the value table converges and the greedy policy over it becomes a path. It needs no map — it discovers the environment's properties itself — which is exactly its selling point and exactly its cost.

2. Canonical formulas

MDP for path planning: **Agent** = the robot; **Environment** = grid/map/simulated world with start, goal and obstacles; **State** = the robot's (x, y) ; **Action** = Up/Down/Left/Right; **Reward** = large positive at the goal, large negative for a collision or leaving the grid, small negative per step (this is what makes it prefer short paths).

Q-learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

α = learning rate; γ = discount factor; the bracket is the temporal-difference error. Bellman optimality, which the table converges to:

$$Q^*(s, a) = r(s, a) + \gamma \max_{a'} Q^*(s', a')$$

Policy: $\pi(s) = \arg \max_a Q(s, a)$.

3. Algorithm

Algorithm

```
QLearning(episodes, alpha, gamma, epsilon):
1.  initialise Q(s,a) = 0 for every state-action pair
2.  for each episode:  start at s; while s is not terminal:
3.      choose a:  random with probability epsilon (EXPLORE), else argmax
        Q(s,.) (EXPLOIT)
4.      take a, observe reward r and next state s'
5.      Q(s,a) = Q(s,a) + alpha * (r + gamma * max_a' Q(s',a') - Q(s,a)); s =
        s'
6.  after training, the path is:  from start, repeatedly take argmax Q(s,.)
```

4. Worked example — 1-D corridor, three episodes, by hand

Exam Tip / Worked Example

Setup. States 0,1,2,3,4 in a line. Actions L and R. State 4 is the goal. Reward +10 for entering state 4, -1 for every other step. $\alpha = 0.5$, $\gamma = 0.9$. All Q start at 0.

Episode 1 — the agent walks straight to the goal: R, R, R, R.

s	a	r	$\max_{a'} Q(s', \cdot)$	new $Q(s, a)$	working
0	R	-1	0	-0.500	$0 + 0.5(-1 + 0.9(0) - 0)$
1	R	-1	0	-0.500	
2	R	-1	0	-0.500	
3	R	+10	0 (terminal)	5.000	$0 + 0.5(10 + 0 - 0)$

Only the last state learned anything useful. This is the defining property of Q-learning: value propagates **backwards, one step per episode**.

Episode 2 — identical walk.

s	a	r	$\max_{a'} Q(s', \cdot)$	new $Q(s, a)$	working
0	R	-1	0	-0.750	$-0.5 + 0.5(-1 + 0 + 0.5)$
1	R	-1	0	-0.750	
2	R	-1	5.000	1.500	$-0.5 + 0.5(-1 + 0.9(5) + 0.5)$
3	R	+10	0	7.500	$5 + 0.5(10 - 5)$

The goal's value has now reached state 2. Check row 3 by hand: $-0.5 + 0.5(-1 + 0.9(5.0) - (-0.5)) = -0.5 + 0.5(-1 + 4.5 + 0.5) = -0.5 + 2.0 = 1.5$. ✓

Episode 3 — with one exploratory wrong move: R, L, R, R, R, R.

s	a	r	$\max_{a'} Q(s', \cdot)$	new $Q(s, a)$
0	R	-1	0	-0.875
1	L	-1	0	-0.500
0	R	-1	-0.500	-1.163
1	R	-1	1.500	-0.200
2	R	-1	7.500	3.625
3	R	+10	0	8.750

Q-table after three episodes, and the greedy policy:

s	$Q(s, L)$	$Q(s, R)$	greedy	$Q^*(s, R)$ (true)
0	0.000	-1.163	L (wrong!)	4.580
1	-0.500	-0.200	R	6.200
2	0.000	3.625	R	8.000
3	0.000	8.750	R	10.000

Three things this example is designed to show. (i) *Value propagates backwards one state per episode* — after three episodes the information has travelled from state 3 to state 1 and no further. (ii) *The policy is still wrong at state 0*: because the untried action L has $Q = 0$ and the tried action R has accumulated negative reward, greedy selection picks L. This is the **exploration problem** — “in large environments the agent might never randomly stumble upon the goal”. It is fixed by ϵ -greedy exploration, optimistic initialisation, or reward shaping, not by more greedy episodes. (iii) *Compare with the converged values*. $Q^*(3, R) = 10$, $Q^*(2, R) = -1 + 0.9(10) = 8$, $Q^*(1, R) = -1 + 0.9(8) = 6.2$, $Q^*(0, R) = -1 + 0.9(6.2) = 4.58$. After three episodes we are at 8.75, 3.625, -0.2, -1.163 — still far away. **This is why RL needs thousands of episodes**, and that gap is the honest answer whenever an exam asks about RL's training cost.

5. Compared with

Comparison

	Supervised-learning planner	RL planner
Learning method	learns from a pre-existing dataset of “correct” paths	learns through active trial-and-error exploration
Data requirement	a large, high-quality dataset of expert examples	a reward function and a simulator
Main challenge	performance is capped by the quality and variety of the training data	can be very slow to train; learning from scratch is hard
Best for	problems where optimal paths are easy to compute and data is abundant	complex, unknown, or changing environments

Against evolutionary planners: evolutionary methods plan from scratch every time with no memory of past solutions and search with a population; RL builds a *policy* through experience and reuses that knowledge across scenarios as a single improving agent. Use evolutionary when the environment is static or slowly changing, explicit optimisation is needed, and training data is limited. Use learning when the environment is highly dynamic, lots of interaction data is available, real-time reactive behaviour is needed, and patterns can genuinely be learned from experience.

Against A*/RRT*: RL needs no map and reacts in microseconds once trained, but offers no completeness or optimality guarantee and no way to certify a specific path before executing it.

Limitation / Warning

Advantages: adaptability to unknown or dynamic environments; no explicit map needed; handles high-dimensional state spaces (an arm with many joints).

Disadvantages: *high training cost* — thousands to millions of episodes; *the exploration problem* — in a large environment the agent may never stumble on the goal; *the sim-to-real gap* — a policy that is perfect in simulation can fail on hardware because of unmodelled physics.

6. Best suited when

The environment is unknown or changes faster than a map can be maintained, a simulator exists to train in, and the inference-time reaction speed matters more than a guarantee.

EC3 Q2(c) — modify the scenario so DRL becomes necessary, and design a multi-objective reward

Modified scenario. The rescue robot is one of a fleet operating over a multi-day recovery, entering aftershock-prone floors where debris shifts continuously, smoke density changes minute to minute, and the pre-disaster map is worthless. There is no stable graph to run ACO on and no stable map to run A* on; but the fleet generates thousands of hours of interaction data, and a high-fidelity structural simulator exists. *That combination — no reliable map, abundant interaction data, and a need for millisecond reactions to a shifting floor — is what makes DRL necessary rather than merely possible.*

Multi-objective reward.

$$R = w_1 R_{goal} + w_2 R_{step} + w_3 R_{collision} + w_4 R_{risk} + w_5 R_{info} + w_6 R_{energy}$$

Component	Sign / magnitude	Purpose
R_{goal}	+100 on reaching a victim	the terminal objective; large enough to dominate any accumulated step cost
R_{step}	-1 per timestep	makes the agent prefer short routes; without it, wandering is free
$R_{collision}$	-100 on contact	hard safety constraint; must exceed the goal reward in magnitude, so no route “pays” for a collision
R_{risk}	$-\lambda \cdot$ structural instability of the cell	aftershock-specific: discourages routes under compromised beams even when they are shorter
R_{info}	$+\mu \cdot$ newly observed area	the victim’s exact location is unknown, so exploration must be rewarded, not just exploitation
R_{energy}	$-\nu(e_t \Delta d + e_r \Delta \theta)$	a battery-limited robot that strands itself mid-building is a second casualty

Justification to state: the components are deliberately in tension — R_{info} pushes outward while R_{step} and R_{energy} pull inward, and R_{risk} opposes the shortest route. A single scalar objective (path cost) cannot express that trade-off, which is exactly why the reward is multi-objective and why the weights encode mission policy: raise w_3 and w_4 when the structure is unstable, raise w_5 early in a search when the victim’s position is uncertain. **Algorithms:** PPO, SAC or TD3 learn navigation policies directly from raw LiDAR/camera input, are trained in simulation, and are then transferred to the real robot.

7. ML / DNN / RL

Architecture / Implementation

The escalation from tables to networks. A Q-table is $|S| \times |A|$ entries, so it dies as soon as the state is continuous or high-dimensional. **Deep RL** replaces the table with a network, so the state can be a raw LiDAR scan or camera frame — PPO, SAC and TD3 are the deck’s named algorithms, trained in simulation and transferred to hardware.

Hybrid paradigms the deck lists:

- **Hierarchical:** a graph planner (A^*) for the coarse global path, then an evolutionary or learned method for fine-grained trajectory optimisation with kinodynamic constraints — fast global planning plus smooth, feasible local paths.
- **Evolutionary $\rightarrow$ learning:** use evolutionary algorithms to *generate training data* for supervised learning; evolve network architectures (neuroevolution) for planning; learn cost functions or heuristics that then guide the evolutionary search.
- **Dynamic-environment handling:** re-planning triggers (current path infeasible, a significantly better path available, predicted collision within horizon T); velocity-obstacle integration that penalises paths entering predicted collision cones; warm-starting from the previous solution; *anytime* behaviour that returns the best solution so far when time runs out.

Integration — what path planning consumes and produces

From localization: the current pose (x, y, θ) with its covariance. If localization fails, the planner must request re-localization rather than plan from a wrong location.

From mapping: the occupancy grid or costmap — resolution, dimensions, origin, data array

(0 free, 100 occupied, 50 unknown). The global planner uses the static global map; the local planner uses the dynamic local costmap with inflation.

To motion control: a waypoint sequence from the global planner; velocity commands (v, ω) at 10–50 Hz from the local planner. Paths must be *kinodynamically feasible* — respect turning radius, max velocity, max acceleration.

Bidirectional with SLAM: the planner receives map and pose, and can supply *exploration goals* back to SLAM. In **active SLAM** the planner optimises to reduce map uncertainty — which is the same objective as active localization, one level up.

Update rates through the stack: perception 10–100 Hz; localization/SLAM 10–50 Hz; global planning 0.1–1 Hz; local planning 10–50 Hz; motion control 50–200 Hz.

10 Errata — slide values that do not survive recomputation

Every numerical result in this document was recalculated independently. The list below records each place where the course deck’s printed value differs from the recomputed one, what the correct value is, and whether the slide’s *conclusion* still holds. Nothing here is a criticism of the material — these are transcription and rounding artefacts of the kind that accumulate in a 169-slide deck. The point is only that you should not carry a number into an exam without being able to re-derive it.

Likelihood field — endpoint score table

Slide says	$P = 0.96$ at $d = 0.08$ m, with “assumed $\sigma = 0.1$ ”
Recomputed	$P = \exp(-0.0064/0.02) = 0.726$ at $\sigma = 0.1$. The printed score column is self-consistent only for $\sigma \approx 0.25$.
Effect	The formula $P = \exp(-d^2/2\sigma^2)$ is correct; only the stated σ conflicts with the printed scores. The ranking of the five points is unchanged. <i>Always show the exponent.</i>

MCL — particle belief row

Slide says	$bel = (0.8805, 0.0013, 0.1132, 0.0031)$
Recomputed	$bel = (0.8812, 0.0030, 0.1128, 0.0030)$
Effect	Two issues. (i) z_{exp} was rounded $3.6056 \rightarrow 3.6$ before weighting. (ii) 0.0013 is a typo — P_2 and P_4 are symmetric and must carry equal belief; the printed four values sum to 0.998, not 1. Resampling outcome unchanged.

EKF-SLAM — data association

Slide says	$d_{L_1}^2 = 0.0147$
Recomputed	$d_{L_1}^2 = 0.5^2/6 = 0.0417$
Effect	$d_{L_2}^2 = 20.25/6 = 3.375$ is printed correctly, which confirms $S = 6$; therefore $d_{L_1}^2$ must be $0.25/6$. L_1 is still selected — conclusion unchanged.

GraphSLAM — solved state vector

Slide says	$y' = [0, 2.11, 3.18, 4.45, 9.18]$, giving $F(y') = 0.0475$
Recomputed	$y^* = [0, 2.0, 3.1, 4.4, 9.1]$, giving $F(y^*) = 0.030$
Effect	The slide value is one damped Gauss–Newton step, not the converged least-squares optimum (its F is higher). Both are defensible in an exam <i>provided you state which you computed</i> .

Firefly — attractiveness β

Slide says	$\beta \approx 0.9$ for the move $(2, 5) \rightarrow (3, 2)$, giving $x^{new} = (2.86, 2.26)$
Recomputed	$r^2 = 10$, so $\beta = e^{-0.1(10)} = 0.3679$, giving $x^{new} = (2.328, 3.856)$
Effect	$\beta = 0.9$ would require $r^2 \approx 1.05$, not 10. The <i>mechanism</i> — move a fraction β toward the brighter firefly, then add $\alpha(\text{rand} - 0.5)$ — is unaffected.

Metaheuristic way-point path lengths (Firefly / GA / ACO / PSO slides)

Slide says	$L_1 = 12.65, \quad L_2 = 14.02, \quad L_3 = 12.36$
Recomputed	$L_1 = 13.34, \quad L_2 = 14.22, \quad L_3 = 13.34$
Effect	Segment-by-segment audit below. This one matters: the corrected lengths make P_1 , P_2 and P_3 nearly <i>equal</i> , so the slides' fitness values, roulette shares and “iteration best” figures all shift. The worked examples in Section 10's preceding chapter avoid these numbers entirely.

Logistic-regression mapping — sweep 3 label

Slide says	“ $P(\text{map}) = \text{sigmoid}(0.1) = 0.986$ ”
Recomputed	$\text{sigmoid}(4.3) = 0.9866$
Effect	A label carried over from sweep 2. The printed <i>value</i> is right; the argument shown with it is not.

D* worked slides — the $h(n)$ table

Slide says	$h(1, 1) = 13.45, h(9, 9) = 1.41, h(8.5, 8.5) = 2.38, h(5.5, 5.5) = 7.78$
Recomputed	Euclidean to $(10.5, 10)$: 13.09, 1.80, 2.50, 6.73
Effect	The slide flags this itself — “ <i>the heuristic is only an assumed dummy value</i> ”. The PRM slides on the identical geometry use the correct Euclidean values. Section 6 of these notes uses Euclidean h throughout, so the A* and D* traces there are internally consistent.

Segment audit of the way-point paths

The four candidate paths used across the Firefly, GA, ACO and PSO slides, with every segment recomputed:

Path	Segments (recomputed)	True L	Slide L
P_1 (1,1), (3,2), (6,4), (9,8), (10.5,10)	2.2361 + 3.6056 + 5.0000 + 2.5000	13.342	12.65
P_2 (1,1), (2,5), (5,6), (8,9), (10.5,10)	4.1231 + 3.1623 + 4.2426 + 2.6926	14.221	14.02
P_3 (1,1), (4,3), (6.5,7), (9.5,9), (10.5,10)	3.6056 + 4.7170 + 3.6056 + 1.4142	13.342	12.36
P_4 (1,1), (3,4), (6,6), (9,9), (10.5,10)	3.6056 + 3.6056 + 4.2426 + 1.8028	13.257	13.26 ✓

P_4 is the only one that checks out. For P_1 the printed final segment is 1.80, but $(9, 8) \rightarrow (10.5, 10)$ is $\sqrt{1.5^2 + 2^2} = 2.50$ — the 1.80 belongs to P_4 's final segment $(9, 9) \rightarrow (10.5, 10)$. For P_3 three of the four segments are wrong. Note that the ACO slide computing Ant 5 uses 2.50 for the same final leg, so the deck is internally inconsistent as well.

How to use this list in the exam

Do not spend time in the hall reconciling a slide number. Instead:

1. **Write the formula before substituting.** A correct formula with an arithmetic slip earns most of the marks; a bare number that disagrees with the examiner's key earns none.
2. **State every assumption you had to supply** — “ Q is not given; I take $Q = 1$ ”, “I take $\sigma = 0.25$ ”, “I treat Z_j as ranges”. The EC3 paper omitted Q deliberately.
3. **Sanity-check totals.** Beliefs sum to 1; a covariance stays positive; a normalised weight

lies in $[0, 1]$; a path length is at least the straight-line distance. These take five seconds and catch most errors.

4. **Say what your answer means.** Every worked example above ends with one sentence of interpretation, because “0.881” scores less than “ P_1 holds 88% of the belief and P_2 will most likely be resampled out”.

Sources

- `AIMLRobots Watemarked.pdf` — 169 slides, the official course deck (Prof. Raja Vadhana Prabhakar and Dr. Sangeetha Viswanathan).
- 17 recorded contact-session transcripts, `AI and ML for Robotics (S2-25_AIMLCZG528)*.docx`, 25 Apr – 29 Aug 2026. (Files (7) and (16) are duplicates of the 25 Apr session.)
- `AIMLEndSemRegular2026.docx` — the EC3 end-semester paper, rescue-robot use case.
- Prescribed reading referenced in the deck: T1 chapters 4, 5, 6, 7, 8; T2 chapters 5, 6.1, 6.2, 12.